

E.T.S. de Ingeniería Industrial,
Informática y de Telecomunicación

Adaptación del algoritmo FARC-HD a Python: optimización y análisis de rendimiento



Grado en Ingeniería Informática

Trabajo Fin de Grado

Autor: Iñaki Mellado Ilundain

Director: José Antonio Sanz Delgado

Pamplona, enero de 2026

upna

Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa

Agradecimientos

Parece ayer, pero han pasado 4 años y medio desde que comencé el grado de ingeniería informática. Inicialmente sin conocer a nadie, indeciso y cambiando completamente de rama de estudios. Hoy puedo decir que he disfrutado de este camino y puedo llamarme a mi mismo ingeniero informático. Sin embargo, no todo ha sido únicamente mérito mío, ya que especialmente al principio tuve serias dudas sobre si había tomado la decisión correcta. Por ello, quiero agradecer y dedicar este trabajo a todas aquellas personas que han estado a mi lado durante este recorrido.

Los primeros y más importantes han sido mis padres y mi hermano. Ellos han vivido en primera persona mis mayores logros y también mis derrotas a lo largo de estos años, pero siempre han estado ahí: unas veces alegrándose conmigo y otras ayudándome a olvidar mis frustraciones.

Siguiendo con los agradecimientos, no puedo olvidarme del increíble grupo de compañeros que he tenido durante la carrera. Ellos son una de las principales razones por las que he logrado terminar el grado con éxito. Como he mencionado, comencé este camino solo y con un plan de futuro que dio un giro de 180 grados: pasé de prepararme para estudiar el grado de Veterinaria en Canarias, con mi primera experiencia viviendo de manera semiautónoma, a quedarme en Pamplona estudiando Informática, dos grados completamente distintos (poco tienen que ver los animales con los ordenadores). Por todo ello, se merecen ser mencionados con nombre y apellido: Lucía Azcona, Asier Larrayoz, Haizea Moral, Beñat Iribarren, Ángel Pérez, Miguel Milagros, Joseba Juanotena, Markel Álvarez, Pablo Labat, Unai Pérez e Iñaki Cerezo.

También se merece una mención especial José Antonio Sanz, primero como profesor y, sobre todo, por guiarme durante el desarrollo de este proyecto, que ha tenido una duración de dos años. Lo que comenzó como una beca de colaboración ha terminado convirtiéndose en mi Trabajo Fin de Grado y, sin duda, ha sido uno de los trabajos que más he disfrutado a lo largo de la carrera, tanto por la parte de investigación necesaria para complementar mis conocimientos como por el propio desafío que ha supuesto el proyecto.

Por último, quiero agradecer a la UPNA como institución y al conjunto de profesores que conforman el grado en Ingeniería Informática. Gracias a ellos cuento con las herramientas, los conocimientos y el prestigio necesarios para comenzar la nueva etapa que supone el mundo laboral.

Resumen

Este trabajo consiste en la implementación del clasificador difuso FARC-HD en el lenguaje Python, a partir de su versión original desarrollada en Java. El objetivo principal es analizar y optimizar el rendimiento del algoritmo mediante la aplicación de distintas técnicas de mejora del tiempo de ejecución.

Para ello, se van a llevar a cabo una transcripción inicial idéntica al código original, seguida de una fase de optimización centrada en la reducción de la carga computacional, el uso eficiente de estructuras de datos y la exploración de herramientas de aceleración en Python.

Finalmente, se comparan los resultados obtenidos con la versión inicial para evaluar el impacto de las mejoras propuestas en términos de eficiencia y mantenibilidad del código.

Lista de palabras clave

FARC-HD, Sistemas de Clasificación Basados en Reglas Difusas (SCBRDs), algoritmos genéticos, Reglas de asociación difusas, vectorización, indexación booleana

Alineación con los Objetivos de Desarrollo Sostenible (ODS)

El presente Trabajo Fin de Grado, centrado en la implementación y optimización del algoritmo FARC-HD en Python, se alinea estratégicamente con la Agenda 2030 de las Naciones Unidas. El desarrollo de software eficiente y accesible es un pilar fundamental para el progreso tecnológico sostenible. En concreto, este proyecto contribuye de manera directa a los siguientes Objetivos de Desarrollo Sostenible:

- **ODS 9: industria, innovación e infraestructura.** Este trabajo tiene el objetivo de promover la innovación tecnológica. Al realizar la migración del algoritmo FARC-HD desde Java a Python (lenguaje predominante en la ciencia de datos actual), se moderniza una infraestructura tecnológica clave en el campo de la Inteligencia Artificial Difusa. La optimización del rendimiento mediante técnicas avanzadas como la vectorización y la compilación JIT con Numba no solo mejora la velocidad de ejecución, sino que permite que estos sistemas sean viables en entornos industriales reales con recursos limitados. Esto favorece la adopción de sistemas de clasificación interpretables (XAI) en la industria, permitiendo tomar decisiones automatizadas transparentes y auditables, un requisito cada vez más demandado en sectores críticos como la medicina o las finanzas.
- **ODS 4: educación de calidad.** El proyecto tiene un fuerte impacto educativo al eliminar barreras de entrada para el aprendizaje de técnicas avanzadas de Inteligencia Artificial. La implementación original en Java, aunque eficiente, presentaba una curva de aprendizaje elevada para estudiantes e investigadores noveles. Al ofrecer una versión de código abierto en Python, se facilita el acceso a materiales didácticos sobre lógica difusa y algoritmos evolutivos. Esto permite que estudiantes universitarios puedan experimentar, modificar y comprender el funcionamiento interno del clasificador, fomentando la adquisición de competencias técnicas esenciales para el mercado laboral actual y promoviendo una educación técnica superior más práctica y accesible.

Este proyecto aborda la sostenibilidad ambiental de manera transversal, estableciendo una relación directa entre la calidad del código y la responsabilidad ecológica. La optimización algorítmica actúa como la causa fundamental para el cumplimiento del **ODS 7 (Energía asequible y no contaminante)**, ya que la reducción de la carga computacional y los tiempos de ejecución disminuye directamente la demanda eléctrica de los procesadores. Esta eficiencia energética tiene como consecuencia una contribución al **ODS 13 (Acción por el clima)**, pues un menor consumo de electricidad deriva en una reducción de la huella de carbono digital y de las emisiones de gases de efecto invernadero asociadas (si la electricidad es obtenida mediante materias primas convencionales).

Contenido

Agradecimientos	1
Resumen.....	3
Lista de palabras clave	3
Alineación con los Objetivos de Desarrollo Sostenible (ODS).....	4
1. Introducción	6
1.1 Objetivo general.....	8
1.2 Objetivos específicos.....	8
2. Marco teórico	10
2.1 Machine Learning.....	10
2.2. Modelos de clasificación	11
2.3. Lógica difusa	17
2.3.1 Elementos de la lógica difusa	17
2.3.2. Sistemas de Clasificación basados en reglas difusas (SCBRD).....	20
2.4. Algoritmos genéticos.....	23
3. Algoritmo FARC-HD	29
3.1. Introducción	29
3.2. Fases del algoritmo	30
3.2.1 Extracción de reglas de asociación difusas de clasificación	30
3.2.2 Preselección de reglas candidatas.....	32
3.2.3 Selección genética de reglas y ajuste lateral de las funciones de pertenencia	33
4. Entorno de experimentación	36
4.1 Entorno de desarrollo	36
4.2 Conjunto de datos	36
5. Optimización y análisis de rendimiento	38
5.1 Implementación base (versión pura en Python).....	39
5.2 Uso de máscaras de datos.....	44
5.3. Optimización mediante Vectorización: Eliminación de Bucles en Soporte y FRM	46
5.4. Compilación JIT con Numba	49
5.5 Evaluación global del rendimiento.....	54
6. Conclusiones y trabajo futuro	57
6.1 Conclusiones Generales	57
6.2 Líneas de Trabajo Futuro.....	58
8. Bibliografía y referencias.....	60

1. Introducción

Hoy en día, los datos se han convertido en un recurso esencial para la sociedad y la economía, comparable en importancia a materias primas o fuerza de trabajo, aunque de un valor intangible y difícil de cuantificar. La información generada por empresas, instituciones, personas, maquinaria... constituye un activo estratégico que permite tomar decisiones informadas y anticipar comportamientos futuros. Desde el seguimiento de tendencias de consumo hasta la predicción de fenómenos complejos, los datos se han transformado en el combustible que impulsa la innovación tecnológica y la competitividad. Su correcta recolección, organización y análisis es clave para aprovechar todo su potencial y convertirlo en conocimiento útil y aplicable.

El análisis y la clasificación de estos datos se ha convertido en un área fundamental dentro del aprendizaje automático [16] y la inteligencia artificial, debido a su amplia aplicación en campos como la medicina, la industria y las finanzas, e incluso en disciplinas creativas (tradicionalmente consideradas exclusivamente como humanas) como: la escritura, la música, la pintura o la generación de videos. El aprendizaje automático [16] es un área de la inteligencia artificial centrada en desarrollar algoritmos capaces de aprender de los datos y mejorar su rendimiento con la experiencia, sin necesidad de ser programados explícitamente para cada tarea [1]. Este campo abarca una amplia variedad de técnicas. Los sistemas de clasificación son un tipo particular de aprendizaje supervisado que permite asignar etiquetas o categorías a nuevas instancias basándose en patrones previamente aprendidos a partir de datos históricos [2].

Los sistemas de clasificación permiten organizar grandes volúmenes de información y tomar decisiones basadas en patrones identificados en los datos. Estas herramientas facilitan, por ejemplo, la predicción de resultados, la automatización de tareas complejas o la extracción de conocimiento útil a partir de conjuntos de datos heterogéneos, lo que evidencia la relevancia del análisis de datos para la resolución de problemas reales. Nos encontramos así en un momento de cambio, en una nueva etapa tecnológica que algunos consideran comparable, en impacto y alcance, a las revoluciones industriales de los siglos pasados: la revolución digital impulsada por la inteligencia artificial.

Sin embargo, muchos métodos tradicionales presentan limitaciones en cuanto a velocidad, precisión e interpretabilidad. La creciente disponibilidad de grandes volúmenes de datos resulta beneficiosa para el entrenamiento de modelos más precisos, pero también exige un mayor poder computacional y estrategias de aprendizaje más eficientes. Por ello, surge la necesidad de algoritmos capaces de generar reglas comprensibles y, al mismo tiempo, mantener un rendimiento competitivo, como es el caso de los clasificadores basados en reglas difusas [3], [4]. Estos sistemas permiten equilibrar la interpretabilidad de los modelos con su capacidad

predictiva, ofreciendo una alternativa que supera algunas de las restricciones propias de los métodos clásicos.

Dentro de los sistemas basados en reglas difusas, se han desarrollado numerosos enfoques que buscan optimizar tanto la precisión como la interpretabilidad del modelo. Estos clasificadores se caracterizan por representar el conocimiento mediante reglas lingüísticas, las cuales constan de uno o varios antecedentes y un consecuente generalmente expresadas en la forma Si (antecedentes) entonces (consecuente), lo que facilita la comprensión del proceso de decisión por parte del usuario. Entre los métodos existentes destacan aquellos que incorporan técnicas evolutivas, de optimización o de reducción de reglas para mejorar la eficiencia del sistema sin comprometer su capacidad predictiva. A diferencia de otros métodos más opacos, este tipo de clasificadores ofrece una ventaja significativa en términos de interpretabilidad: cada decisión tomada por el sistema puede justificarse mediante un conjunto de reglas lingüísticas comprensibles, lo que contribuye a aumentar la confianza del usuario en sus resultados. Este tipo de modelos se enmarca en el paradigma de la eXplainable Artificial Intelligence (XAI) [5], un área de creciente interés que busca dotar a los sistemas de inteligencia artificial de mayor transparencia e interpretabilidad.

En este contexto, el algoritmo FARC-HD sobresale del resto de algoritmos debido a la combinación de la lógica difusa con técnicas de selección evolutiva, lo que permite generar conjuntos de reglas precisas y compactas [4]. Actualmente, FARC-HD se encuentra implementado en la herramienta KEEL, cuyo código está desarrollado en Java. Sin embargo, Python se ha consolidado como el lenguaje predominante en el ámbito del aprendizaje automático, gracias a su amplia comunidad, sus bibliotecas especializadas (Scikit-learn, NumPy o TensorFlow) y su facilidad de uso. Por ello, resulta de gran interés disponer de una implementación de FARC-HD en Python, lo que permitiría su integración con otros entornos de desarrollo, facilitaría su utilización por parte de investigadores y fomentaría la experimentación y extensión del algoritmo en futuras investigaciones.

En este trabajo se implementa FARC-HD en Python y se estudian diversas estrategias de optimización para mejorar su rendimiento, incluyendo la vectorización de datos, la eliminación de bucles y la compilación JIT con Numba [17]. Estas técnicas permiten reducir el tiempo de ejecución y aprovechar las capacidades computacionales modernas sin sacrificar la precisión ni la interpretabilidad del clasificador. De este modo, el presente trabajo busca no solo reproducir fielmente el comportamiento del clasificador original, sino también explorar el potencial de las optimizaciones modernas en Python.

Este documento se organiza de la siguiente manera:

- El capítulo 2 presenta los objetivos del proyecto, incluyendo tanto los generales como los específicos.

- El capítulo 3 revisa el marco teórico, proporcionando la explicación de los conceptos que sustentan el funcionamiento del algoritmo FARC-HD.
- El capítulo 4 describe el algoritmo FARC-HD, detallando paso a paso su funcionamiento.
- El capítulo 5 detalla el entorno de experimentación, presentando todas las herramientas y librerías necesarias para la realización del experimento.
- El capítulo 6 analiza las estrategias de optimización y los resultados obtenidos, desde la primera traducción directa del algoritmo hasta la última versión optimizada.
- El capítulo 7 expone las conclusiones y posibles líneas de trabajo futuro, identificando los aspectos en los que FARC-HD podría mejorar su rendimiento.

1.1 Objetivo general

El objetivo principal de este trabajo es implementar, evaluar y optimizar el algoritmo FARC-HD en Python, garantizando que su ejecución sea correcta y equivalente a la versión original en Java, y mejorando su eficiencia computacional. Este objetivo busca asegurar que el clasificador sea preciso, eficiente e interpretable; manteniendo la exactitud de los resultados originales y empleando distintas técnicas de optimización para alcanzar un rendimiento comparable o superior al de la implementación en Java.

1.2 Objetivos específicos

Para alcanzar el objetivo general, se plantean los siguientes objetivos específicos:

1. Implementar el algoritmo FARC-HD en Python, asegurando que los valores obtenidos coincidan con los resultados de la versión en Java. Este paso permite validar la equivalencia funcional entre ambas versiones.
2. Analizar los cuellos de botella presentes en el programa original que puedan afectar al rendimiento de la traducción en Python, especialmente los bucles simples y bucles anidados
3. Aplicar técnicas de optimización, como vectorización de datos, eliminación de bucles o uso de librerías más específicas para la optimización como Numba [17], cada una de ellas evaluada de manera independiente.
4. Evaluar el impacto de las estrategias de optimización en el tiempo de ejecución en los puntos críticos del programa (generación de reglas y algoritmo genético), asegurando que la precisión del clasificador se mantenga.
5. Analizar la interpretabilidad de las reglas generadas por FARC-HD y su utilidad para justificar las decisiones del sistema.
6. Comparar cuantitativamente las distintas versiones optimizadas del algoritmo y documentar las mejoras alcanzadas en eficiencia y tiempo de

ejecución. Esto permitirá extraer conclusiones sobre qué combinación de técnicas resulta más adecuada para acelerar sistemas basados en reglas difusas.

2. Marco teórico

2.1 Machine Learning

Se conoce como Inteligencia Artificial (IA) [18] al campo de la ingeniería informática encargado de reproducir en máquinas y ordenadores las capacidades humanas, como la resolución de problemas, el razonamiento, la creatividad o la comprensión de textos. El objetivo principal de la IA es que las máquinas sean capaces de percibir su entorno, procesar información y tomar decisiones de forma autónoma, con una intervención humana parcial o incluso nula [8].

Durante la segunda mitad del siglo XX surge un nuevo enfoque dentro de la inteligencia artificial, con algoritmos capaces de aprender a partir de la experiencia en lugar de limitarse a seguir reglas predefinidas. Este cambio de paradigma dio origen al campo del Machine Learning [16]. El objetivo del ML consiste en la búsqueda de patrones en un conjunto de datos de entrenamiento para mejorar la predictibilidad de datos futuros. El cambio con la IA previa se da en que estos patrones no son escritos previamente, sino que son descubiertos por el algoritmo [7].

Desde los primeros trabajos de Alan Turing en los años cincuenta, que sentaron las bases conceptuales de la Inteligencia Artificial, hasta la actual expansión del Machine Learning, este campo ha experimentado una evolución constante (véase Figura 1), marcada por hitos como el desarrollo de los primeros sistemas expertos en los años setenta y ochenta, y el posterior auge del Big Data y del aprendizaje profundo, impulsado por el aumento de la capacidad computacional en las últimas décadas.

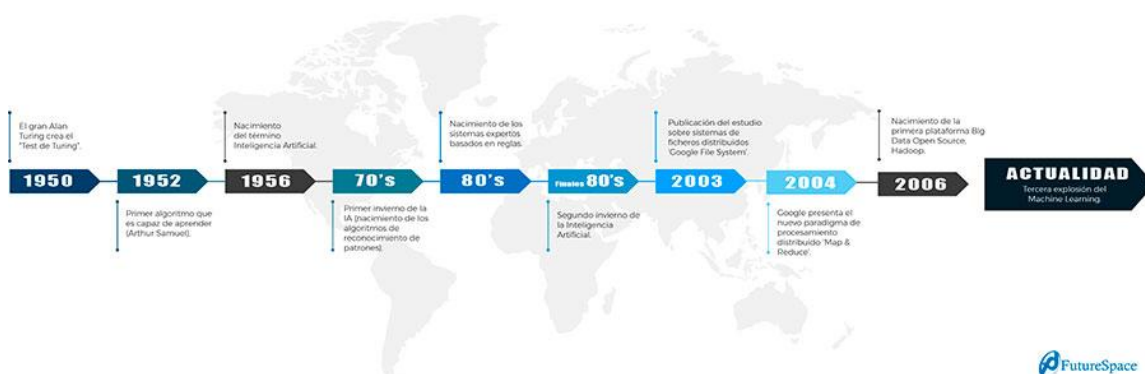


Figura 1: Evolución del machine learning [7]

En el siglo XXI, el aprendizaje automático sigue siendo fundamental en la resolución de una gran diversidad de problemas. Una de sus áreas más destacadas es el Deep Learning, basado en redes neuronales profundas que permite aprender de grandes volúmenes de datos, útiles en tareas como procesamiento de lenguaje natural, visión

por computador y otras aplicaciones donde se requiere detectar relaciones complejas en los datos. Estos avances permiten desarrollar sistemas complejos y precisos [8].

Los algoritmos de aprendizaje automático pueden clasificarse en 4 grandes grupos:

- **Aprendizaje Supervisados:** Requieren de la información que aportan las variables de entrada y de salida para poder obtener los patrones que resuelven el problema. Se requiere por ello supervisión humana (etiquetación previa de los datos) [11]. Algunos ejemplos de algoritmos supervisados son: regresión lineal y logística, los árboles de decisión, las redes neuronales o K-NN [12].
- **Aprendizaje No Supervisados:** a diferencia del aprendizaje supervisado, este no requiere de supervisión humana (no existe una etiquetación previa de los datos) [11]. Algunos ejemplos de algoritmos supervisados son: agrupamiento jerárquico o k-means [12].
- **Aprendizaje semi-Supervisados:** emplean el uso de un pequeño grupo de datos etiquetados que se utilizaran para predecir inicialmente los datos sin etiqueta [13].
- **Aprendizaje por Refuerzo (Reinforcement Learning):** Se basa en la interacción continua entre un agente y un entorno. El agente observa el estado del entorno, toma decisiones para maximizar recompensas y ajusta su comportamiento en función de los resultados obtenidos. A diferencia de los métodos supervisados y no supervisados, no se basa en la búsqueda de patrones en los datos, sino que aprende mediante ensayo y error. Este enfoque ha ganado relevancia en los últimos años [19].

2.2. Modelos de clasificación

Los sistemas de clasificación [20] constituyen una de las ramas fundamentales del aprendizaje supervisado (definido previamente en el punto anterior). Su objetivo principal es construir modelos capaces de asignar una clase conocida a un nuevo ejemplo, a partir de un conjunto de datos previamente etiquetados. Cada clase se designa como $C_j \in C = \{1, \dots, M\}$, donde C representa el conjunto total de M clases discretas (de no ser discretas, nos encontraríamos ante un problema de regresión con valores continuos). En otras palabras, el problema de clasificación consiste en determinar a cuál de esas categorías pertenece un determinado ejemplo $X_i \in X$ con el mínimo error de clasificación posible [3].

Por lo general, el funcionamiento de un sistema de clasificación puede dividirse en dos fases fundamentales: la de entrenamiento y la de predicción. Durante la fase de entrenamiento, el algoritmo analiza un conjunto de ejemplos previamente conocidos y con la clase correctamente etiquetada con el objetivo de construir un modelo que

capture las relaciones existentes entre las variables de entrada y las salidas esperadas. Este proceso se ilustra en la Figura 2: la frontera de decisión se representa en negro, los ejemplos clasificados como clase 1 aparecen en azul y los clasificados como clase 2 en naranja. Durante la fase de predicción, el modelo aplica la función de predicción aprendida $y = f(x)$ sobre nuevos datos para determinar la clase a la que pertenece cada instancia [14]. En la figura, se observa un nuevo ejemplo destacado que ha sido asignado a la clase 2.

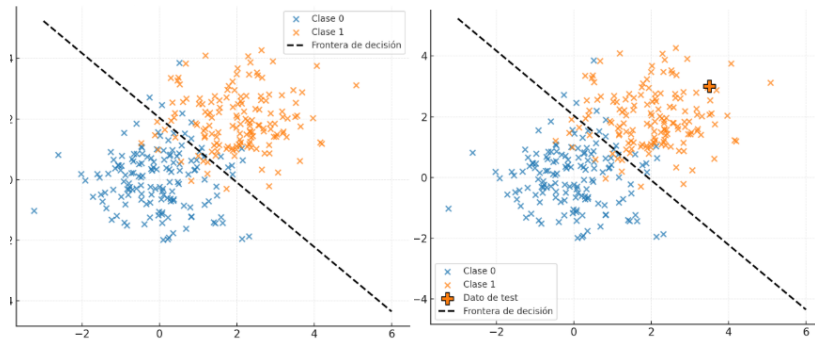


Figura 2: fases de entrenamiento y predicción

La capacidad del modelo para ofrecer resultados precisos sobre datos no etiquetados se conoce como generalización. Un modelo con buena capacidad de generalización es aquel que logra un equilibrio entre la fidelidad al conjunto de entrenamiento y la habilidad para adaptarse a nuevas situaciones, evitando tanto el subajuste (modelo con una frontera de decisión demasiado simple, con un sesgo alto) como el sobreajuste (modelo con una frontera de decisión demasiado compleja, con una varianza alta), tal como se ilustra en la Figura 3 [15].

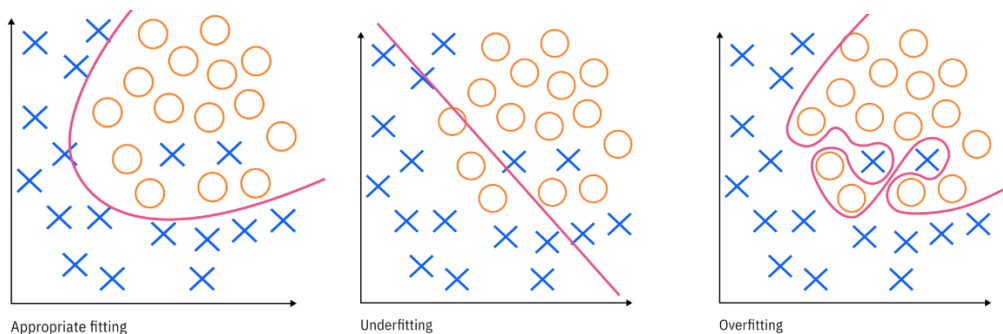


Figura 3: Subajuste y sobreajuste [15]

Para garantizar un aprendizaje adecuado y una evaluación objetiva del modelo, el conjunto de datos disponibles suele dividirse en tres subconjuntos diferenciados:

- El **conjunto de entrenamiento**: utilizado para ajustar los parámetros del modelo.

- El **conjunto de validación**: utilizado para optimizar los hiperparámetros y prevenir el sobreajuste.
- El **conjunto de test**: utilizado para medir la capacidad de generalización del clasificador sobre ejemplos no vistos.

De este modo, los conjuntos de entrenamiento y validación se emplean únicamente durante la fase de entrenamiento, mientras que el conjunto de test se reserva para la fase de predicción, asegurando una evaluación imparcial del rendimiento del modelo.

En numerosos problemas de clasificación, el conjunto de ejemplos no tiene una distribución uniforme entre las distintas clases, es decir, algunas clases están representadas por una cantidad muy superior de ejemplos que otras. Este fenómeno se conoce como desbalance de clases y puede afectar negativamente al rendimiento del modelo, ya que tiende a favorecer la predicción de las clases mayoritarias y desfavorecer el resto de ellas. Para reducir este desbalance, se pueden emplear diferentes técnicas de preprocesamiento de datos sobre el conjunto de entrenamiento (nunca al conjunto de validación ni al conjunto de test por ser datos sintéticos):

- **Oversampling**: consiste en aumentar el número de ejemplos de la clase minoritaria, es decir, aquella que presenta menos instancias o que resulta de mayor interés para la clasificación. Esta ampliación puede realizarse de manera aleatoria (Random Oversampling [25]) o mediante la generación de nuevos ejemplos sintéticos en el entorno de los existentes, como en el método SMOTE (Synthetic Minority Over-sampling Technique) [21].
- **Undersampling**: se basa en la reducción del número de ejemplos pertenecientes a la clase mayoritaria o negativa. Dicha reducción puede efectuarse de forma aleatoria (Random Undersampling [25]) o aplicando criterios más sofisticados, como la eliminación de instancias redundantes o lejanas a la frontera de decisión, mediante técnicas como CNN (Condensed Nearest Neighbor) [22], Tomek Links [23] o One-Sided Selection (OSS) [24].
- **Técnicas híbridas**: combinan estrategias de oversampling y undersampling con el objetivo de equilibrar las clases sin generar un exceso de datos sintéticos ni eliminar información relevante del conjunto original.

Una vez entrenado el modelo aplicando, si es necesario, técnicas de balanceo de clases, se procede a evaluar su rendimiento mediante métricas cuantitativas que permitan valorar la calidad de las predicciones y comparar distintos clasificadores. Esta valoración puede realizarse sobre cualquiera de los tres conjuntos de datos (entrenamiento, validación o test), aunque la evaluación final del modelo debe efectuarse sobre el conjunto de test, al ser el único compuesto por ejemplos completamente nuevos y no utilizados durante el aprendizaje. Existen dos grandes grupos de métricas de rendimiento:

- Los basados en la matriz de confusión, que dependen del umbral de clasificación aplicado.
- Los basados en curvas independientes del umbral, que ofrecen una evaluación global del modelo considerando todos los posibles valores de dicho umbral.

Las métricas basadas en la matriz de confusión (similar a la que se muestra en la parte izquierda de la figura 4) se obtienen a partir de una tabla que resume las predicciones del modelo frente a las clases reales. En un problema de clasificación binaria, la matriz de confusión está compuesta por cuatro valores:

- Verdaderos positivos (VP), representan la cantidad de ejemplos correctamente clasificados de la clase positiva.
- Falsos positivos (FP), representan la cantidad de ejemplos de la clase negativa que han sido clasificados como de clase positiva.
- Verdaderos negativos (VN), representan la cantidad de ejemplos correctamente clasificados de la clase negativa.
- Falsos negativos (FN), representan la cantidad de ejemplos de la clase positiva que han sido clasificados como de clase negativa.

En el caso de una clasificación multiclase, la matriz de confusión se amplía para incluir todas las combinaciones posibles entre clases reales y predichas, generando una matriz cuadrada de tamaño $M \times M$, donde M es el número total de clases. Cada elemento de la matriz indica el número de ejemplos cuya clase real pertenece a la categoría i y que han sido clasificados como pertenecientes a la categoría j .

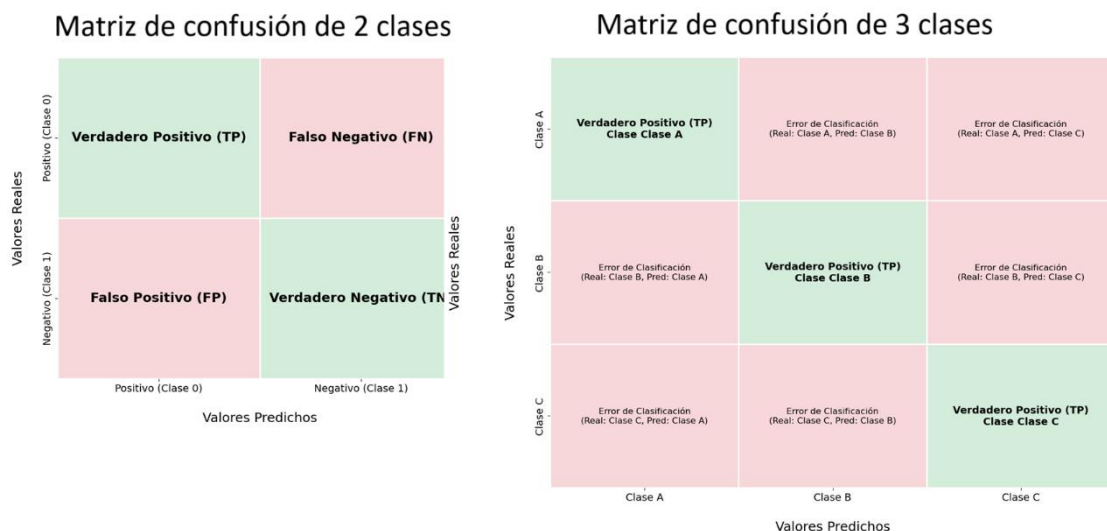


Figura 4: Matrices de confusión de 2 y 3 clases

Para rellenar la matriz de confusión, es necesario establecer un umbral de decisión. Por defecto, Este umbral suele fijarse en 0.5, de modo que, si la probabilidad de

pertenecer a la clase positiva es igual o superior al valor establecido, el ejemplo se clasifica como positivo; en caso contrario, se clasifica como negativo. A partir de los valores contenidos en la matriz de confusión se derivan diversas métricas de rendimiento, entre las que destacan:

- **Accuracy:** mide el porcentaje global de aciertos, tanto positivos como negativos. La forma de calcular el accuracy es:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precisión:** mide la proporción de predicciones positivas que realmente pertenecen la clase positiva. La forma de calcular la precisión es:

$$Precisión = \frac{TP}{TP + FP}$$

- **Recall o TPR:** mide la capacidad del modelo para detectar correctamente las instancias positivas clasificadas correctamente o como negativas. La forma de calcular el Recall es:

$$Recall = \frac{TP}{TP + FN}$$

- **TNR:** mide la capacidad del modelo para reconocer correctamente las instancias negativas. La forma de calcular el TNR es:

$$TNR = \frac{TN}{TN + FP}$$

- **F1-Score:** combina la precisión y la exhaustividad en un único valor mediante su media armónica, útil en escenarios con clases desbalanceadas. La forma de calcular el F1-score es:

$$F1 = 2 \cdot \frac{Precisión \cdot Recall}{Precisión + Recall}$$

- **Recall medio por clases:** mide la capacidad promedio del modelo para identificar correctamente tanto las instancias positivas como las negativas. La forma de calcular el Recall medio es:

$$Recall_{medio} = \frac{TPR + TNR}{2}$$

- **Media geométrica:** mide el equilibrio entre la precisión (TPR) y el TNR. Un valor alto indica que el modelo tiene un rendimiento similar en ambas clases. La forma de calcular la media geométrica es:

$$GM = \sqrt{TPR \cdot TNR}$$

Por otro lado, existen métricas independientes del umbral de clasificación, que evalúan el rendimiento del modelo considerando todos los posibles valores de decisión en lugar de un umbral fijo. Estas medidas ofrecen una visión más completa de las posibilidades del clasificador, resultando estas útiles cuando el coste de los errores varía o cuando se desea comparar modelos con distintos umbrales de decisión. Destacan dos métricas.

La construcción de estas curvas se basa en solicitar al clasificador la probabilidad de pertenencia a la clase positiva para cada instancia, en lugar de simplemente etiquetar como clase positiva o negativa. Posteriormente, se ordenan todas las instancias según su probabilidad (de mayor a menor) y se utiliza cada valor de probabilidad como un umbral de decisión. Para cada uno de estos umbrales, se calcula la matriz de confusión resultante y las métricas asociadas. Este proceso permite construir dos gráficas fundamentales:

- **Área bajo la curva ROC:** representa la relación entre la Tasa de Verdaderos Positivos (TPR o Recall) en el eje Y, y la tasa de Falsos Positivos (FPR) en el eje X tal como se muestra en la sub-figura izquierda de la figura 5. Un valor de 1 en el área bajo la curva indica una clasificación perfecta, mientras que un valor de 0.5 refleja un comportamiento equivalente al azar [26].
- **Área bajo la curva de Precisión–Recall:** representa la relación entre la Precisión (eje Y) y el Recall (eje X) tal como se muestra en la sub-figura derecha de la figura 5. Es una métrica crucial en problemas con clases desbalanceadas, ya que la curva ROC puede ser demasiado optimista en estos escenarios al no ponderar adecuadamente el impacto de los Falsos Positivos sobre la vasta mayoría de negativos [26].

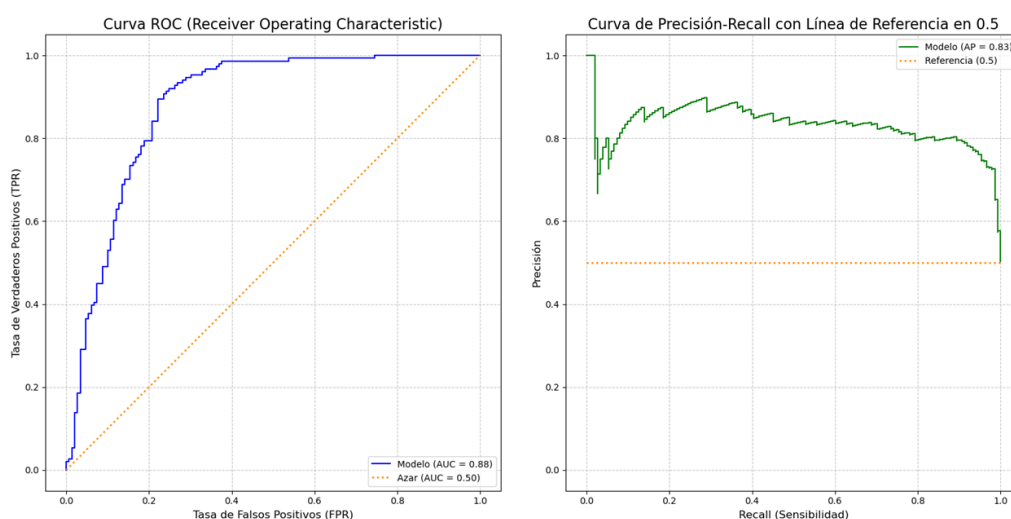


Figura 5: curva izquierda para ROC y derecha para Precision-Recall

2.3. Lógica difusa

La lógica difusa surge como una extensión de la lógica clásica (o booleana), cuyo objetivo es representar de manera más realista los fenómenos que no pueden describirse únicamente mediante valores binarios de verdad (0 para falso o 1 para verdadero). En lugar de limitar las afirmaciones a ser completamente verdaderas o falsas, la lógica difusa permite asignar grados intermedios de pertenencia, con valores continuos. Esta diferencia puede observarse a continuación, en la figura 6, donde se ha simulado dos gráficos que representan que la temperatura del agua es alta: la primera perteneciente a la lógica clásica y la segunda a la lógica difusa.

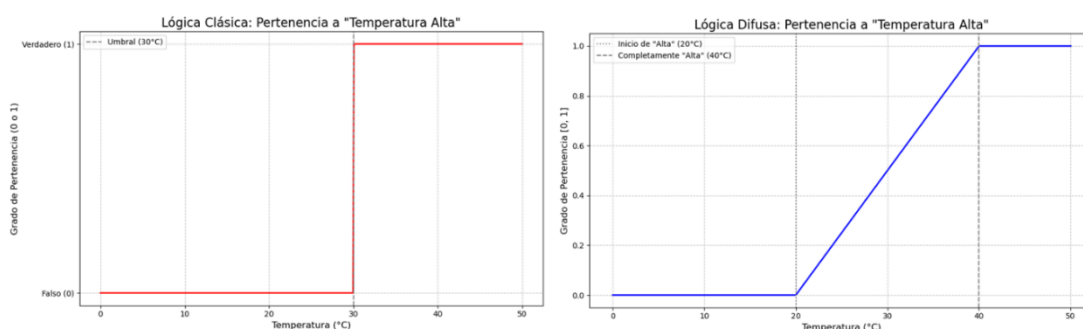


Figura 6: Diferencia entre lógica clásica y difusa

Este enfoque, propuesto originalmente por Lotfi A. Zadeh en 1965 [2][10], ofrece un marco flexible para tratar información imprecisa, ambigua o subjetiva, lo que lo convierte en una herramienta eficaz para modelar el razonamiento humano y, en consecuencia, para el desarrollo de sistemas de clasificación más robustos. Entre sus principales ventajas destacan la interpretabilidad de las reglas, la tolerancia al ruido y la capacidad de generalización ante datos incompletos. Por estas razones, la lógica difusa se ha consolidado como un pilar fundamental en ámbitos como el control inteligente, la toma de decisiones y el aprendizaje automático. Para comprender formalmente este paradigma, es necesario introducir los conceptos fundamentales que lo sustentan.

2.3.1 Elementos de la lógica difusa

Un conjunto difuso [29] se caracteriza por la asignación de un grado de pertenencia a cada elemento del universo de discurso, valor que se representa mediante una función de pertenencia $\mu(x)$ con rango entre 0 (no pertenece al conjunto) y 1 (pertenece completamente al conjunto). A diferencia de los conjuntos clásicos, donde un elemento pertenece o no pertenece al conjunto, en la lógica difusa se permite una pertenencia parcial.

Las funciones de pertenencia son herramientas matemáticas que permiten cuantificar el grado de pertenencia de un elemento a un conjunto difuso. Entre las más utilizadas se encuentran:

- **Función de pertenencia triangular:** Se representa mediante un triángulo definido por tres parámetros: a (inicio), b (pico) y c (fin). Su valor crece linealmente desde a hasta b y decrece linealmente hasta c . Formalmente:

$$\mu(x) = \begin{cases} 0, & x \leq a \\ \frac{x-a}{b-a}, & a < x \leq b \\ \frac{c-x}{c-b}, & b < x < c \\ 0, & x \geq c \end{cases}$$

- **Función de pertenencia trapezoidal:** Es similar a la triangular, pero con un tramo superior plano donde el grado de pertenencia es máximo. Está definido por cuatro parámetros: a (inicio ascensión del trapecio), b (fin ascensión del trapecio), c (inicio descenso del trapecio) y d (fin descenso del trapecio). Formalmente:

$$\mu(x) = \begin{cases} 0, & x \leq a \\ \frac{x-a}{b-a}, & a < x \leq b \\ 1, & b < x \leq c \\ \frac{d-x}{d-c}, & c < x \leq d \\ 0, & x > d \end{cases}$$

- **Función de pertenencia gaussiana:** Esta función adopta una forma de campana suave. Se define mediante el centro c y la desviación estándar σ , controlando la anchura de la curva:

$$\mu(x) = e^{-\frac{(x-c)^2}{2\sigma^2}}$$

Estas funciones permiten traducir información cualitativa a un formato cuantitativo interpretable, constituyendo la base para la construcción de reglas difusas que se emplean en sistemas de clasificación y toma de decisiones.

Las operaciones básicas [28] de la lógica clásica, que marcan la relación entre conjuntos, se redefinen en términos de los valores de pertenencia, dando lugar a una aritmética de la incertidumbre:

- **Intersección (AND):** modelada por una función t-norma [30], que cumple:
 - **Conmutatividad:** $t(x, y) = t(y, x)$, para todo $x, y \in [0,1]$.
 - **Asociatividad:** $t(x, t(y, z)) = t(t(x, y), z)$, para todo $x, y, z \in [0,1]$.
 - **Monotonía:** si $x \leq y$ entonces $t(x, z) \leq t(y, z)$, para todo $z \in [0,1]$.
 - **Condiciones de frontera:** $t(x, 0) = 0$ y $t(x, 1) = x$, para todo $x \in [0,1]$.

- **Unión (OR):** modelada por una función t-conorma [30], dual de la t-norma mediante la dualidad de Morgan: $s(x, y) = 1 - t(1 - x, 1 - y)$, $t(x, y) = 1 - s(1 - x, 1 - y)$, que cumple:
 - **Conmutatividad:** $s(x, y) = s(y, x)$, para todo $x, y \in [0, 1]$.
 - **Asociatividad:** $s(x, s(y, z)) = s(s(x, y), z)$, para todo $x, y, z \in [0, 1]$.
 - **Monotonía:** si $x \leq y$ entonces $s(x, z) \leq s(y, z)$, para todo $z \in [0, 1]$.
 - **Condiciones de frontera:** $s(x, 0) = x$ y $s(x, 1) = 1$, para todo $x \in [0, 1]$.
- **Negación:** extensión del operador lógico **NOT**, modelada mediante una función de negación [30]. Esta función debe ser continua y estrictamente decreciente, cumpliendo:
 - **Condiciones de frontera:** $N(0) = 1$ y $N(1) = 0$.
 - **Monotonía:** $x \leq y \Rightarrow N(x) \geq N(y)$.

En la Figura 7 se ilustra de forma visual el comportamiento de los operadores lógicos difusos fundamentales. Partiendo de dos conjuntos difusos solapados se aplican las tres operaciones básicas: la t-norma modelada utilizando el mínimo, la t-conorma modelada utilizando el máximo y la negación modelada utilizando el complemento.

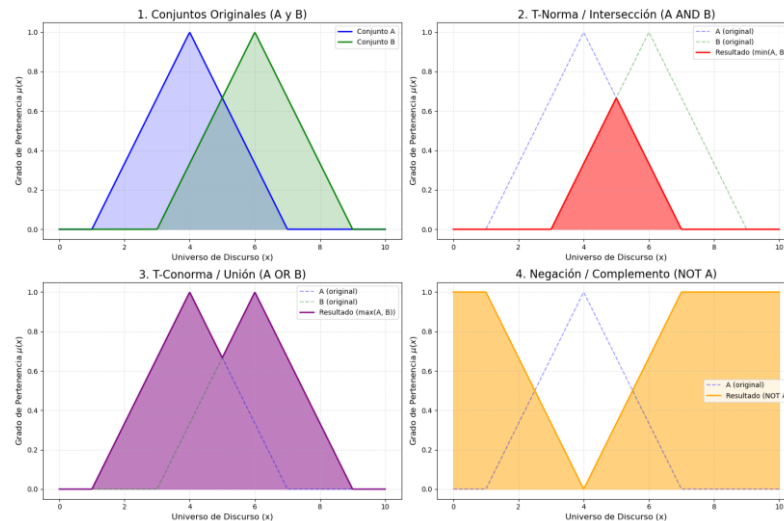


Figura 7: comportamiento de t-normas, t-conormas y negación

Por otro lado, las variables lingüísticas (como temperatura, humedad o altura) representan los atributos que queremos describir en términos cualitativos. Cada una de estas variables puede tomar etiquetas lingüísticas (como baja, media o alta) que se asocian a conjuntos difusos mediante sus correspondientes funciones de pertenencia. Esta representación permite traducir el conocimiento experto a un formato computable, preciso e interpretable semánticamente. Además, existen matizadores lingüísticos

(como muy, poco o casi) que modifican la intensidad de las propiedades representadas por las funciones de pertenencia.

El uso combinado de funciones de pertenencia, operadores difusos y elementos lingüísticos (variables, etiquetas y matizadores) permite construir sistemas difusos capaces de capturar conocimiento experto de manera estructurada y adaptable, estableciendo la base para la definición de reglas difusas que se emplearán en clasificadores y sistemas de decisión.

En la Figura 8 se muestran los elementos fundamentales de la lógica difusa: una variable lingüística, denominada Temperatura; sus etiquetas lingüísticas, llamadas Baja, Media y Alta; y un matizador lingüístico, representado por Muy Alta, que se obtiene aplicando el modificador cuadrado a la función de pertenencia de la etiqueta Alta. Esta representación permite visualizar cómo las variables, etiquetas y matizadores interactúan para traducir conceptos cualitativos en un formato cuantitativo interpretable.

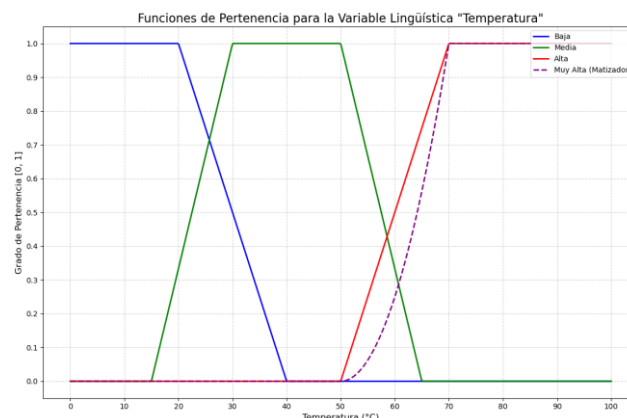


Figura 8: funciones de pertenencia de la variable lingüística Temperatura

2.3.2. Sistemas de Clasificación basados en reglas difusas (SCBRD)

Los Sistemas de Clasificación basados en Reglas Difusas (SCBRD) [3] constituyen un tipo de clasificador que combina la lógica difusa con el aprendizaje supervisado para asignar etiquetas a nuevos ejemplos. A diferencia de los clasificadores clásicos, que generan fronteras de decisión rígidas, los SCBRD permiten manejar incertidumbre e imprecisión en los datos, ofreciendo además una interpretabilidad directa, ya que las decisiones pueden explicarse mediante reglas comprensibles por expertos humanos.

La representación del conocimiento en un SCBRD se organiza en una base de conocimiento difuso que consta de dos componentes principales: una base de datos de términos lingüísticos que contiene todas las variables de entrada, etiquetas y modificadores lingüísticos y una base de reglas formada por reglas difusas que relacionan condiciones sobre las variables de entrada con conclusiones o clases de

salida, siguiendo la forma del Modus Ponens Generalizado. Para un problema de dos variables de entrada las reglas tendrían esta forma (se extendería para N variables de entrada):

Si x_1 es A_1 y x_2 es A_2 entonces y es B

donde $x_1, y x_2$ son las variables de entrada, A_1 y A_2 son las etiquetas lingüísticas asociadas a cada variable mediante funciones de pertenencia e y es la variable de salida o clase predicha, representada por la etiqueta B . En las reglas difusas, el operador lógico “y” se implementa mediante una t-norma, mientras que “o” se implementaría con una t-conorma, permitiendo evaluar combinaciones de condiciones difusas.

Para determinar qué reglas son útiles, los SCBRD emplean medidas de interés adaptadas al contexto difuso. Estas métricas son fundamentales para la selección automática de reglas y sirven como base para los algoritmos evolutivos que las optimizan (como ocurre en FARC-HD). Entre las más utilizadas se encuentran el soporte y la confianza, adaptadas al contexto de clasificación difusa.

El soporte indica el grado en que una regla se cumple dentro del conjunto de datos considerando únicamente los ejemplos que pertenecen a la clase objetivo. Esta medida refleja la representatividad de la regla para la clase específica. Una regla con bajo soporte puede considerarse menos representativa o poco fiable, mientras que una con soporte alto se considera muy fiable, ya que se basa en un número significativo de ejemplos de dicha clase. El soporte difuso para reglas de clasificación se define como:

$$Soporte(A \Rightarrow B) = \frac{\sum_{x_i \in Clase\ C_j} \mu_{AB}(x_i)}{|N|}$$

La confianza mide la proporción de ejemplos de la clase objetivo que cumplen la condición de la regla respecto a todos los ejemplos que cumplen la condición. Esta métrica indica cuán fiable es la implicación “Si A entonces C_j ”. Una regla con alta confianza se considera muy fiable, ya que la mayoría de los ejemplos que cumplen el antecedente pertenecen a la clase objetivo, mientras que una regla con baja confianza se considera menos representativa. La confianza difusa para reglas de clasificación se define como:

$$Confianza(A \Rightarrow B) = \frac{\sum_{x_i \in Clase\ C_j} \mu_{AB}(x_i)}{\sum_{x_i \in T} \mu_A(x_i)}$$

Donde:

- x_i : representa el i -ésimo ejemplo del conjunto de datos.
- T : conjunto total de ejemplos o transacciones del dataset.

- C_j : clase objetivo de la regla difusa.
- $|N|$: número total de ejemplos en el conjunto de datos.
- $\mu_A(x_i)$: grado de compatibilidad del ejemplo x_i al antecedente de la regla.
- $\mu_{AB}(x_i)$: grado de compatibilidad del ejemplo x_i al antecedente y consecuente de la regla.

La combinación de soporte y confianza proporciona una evaluación completa de la relevancia de una regla difusa: el soporte garantiza que la regla sea representativa dentro de la clase objetivo, mientras que la confianza asegura que la regla sea precisa y fiable para la predicción. Juntas, estas métricas permiten identificar las reglas más útiles y robustas dentro de un sistema de clasificación difusa. Esta unión será especialmente importante en sistemas como FARC-HD, donde la selección de reglas óptimas es un paso fundamental para construir un clasificador preciso y eficiente, como se explicará con más detalle en la sección dedicada a este sistema.

El modelo de razonamiento difuso utiliza la base de conocimiento para predecir la clase de nuevas instancias mediante un proceso estructurado de inferencia. Este proceso se desarrolla en tres etapas fundamentales:

1. **Fuzzificación:** Los valores numéricos de las variables de entrada se transforman en grados de pertenencia a las etiquetas lingüísticas definidas en la base de datos de términos lingüísticos.
2. **Inferencia:** Cada regla de la base de reglas se evalúa combinando los grados de pertenencia de sus antecedentes mediante los operadores difusos correspondientes (t-normas o t-conormas). Además, se considera el grado de certeza o peso de la regla, que puede adoptarse de tres formas:
 - **Regla sin ponderación:** se aplica con igual confianza a todas las clases.
 - **Regla con grado global de certeza:** un único valor $\alpha \in [0,1]$ que indica la confiabilidad general de la regla.
 - **Regla con grado por clase:** distintos valores de certeza α_i para cada clase posible, reflejando que la regla puede ser más confiable para unas clases que para otras.
3. **Defuzzificación:** Los resultados difusos de todas las reglas activadas se combinan para producir una clase concreta, que representa la predicción del sistema. Un método habitual es el centro de gravedad, que calcula un valor ponderado según los grados de activación de las reglas.

Para la clasificación de nuevos ejemplos, SCBRD emplea un enfoque de razonamiento difuso creado por H. Ishibuchi [31], en el que cada ejemplo se evalúa respecto a la base de reglas mediante su grado de compatibilidad con los antecedentes. Este grado se combina con los grados de certeza de las reglas para obtener un grado de asociación con cada clase, y finalmente se aplica una función de agregación para determinar la clase con mayor fuerza de asociación. Este esquema resume de manera compacta el proceso de inferencia difusa y es el método utilizado en FARC-HD para predecir la clase de nuevas instancias.

La integración de la base de conocimiento y el modelo de razonamiento difuso constituye el núcleo de los clasificadores basados en reglas difusas y sienta las bases para modelos específicos como FARC-HD, un sistema que extrae automáticamente reglas difusas a partir de datos etiquetados y que se utiliza en este trabajo como referencia práctica.

2.4. Algoritmos genéticos

Si bien durante las décadas de los cincuenta y sesenta empiezan a surgir las primeras ideas que más tarde darían lugar a los algoritmos evolutivos, no es hasta finales de los años sesenta cuando comienzan a consolidarse los intentos de formular una teoría general de la adaptación capaz de explicar cómo sistemas complejos —biológicos, económicos o computacionales— mejoran su rendimiento en entornos inciertos. Este enfoque sería desarrollado de forma formal por John Holland, cuyo trabajo en sistemas adaptativos estableció las bases conceptuales de operadores evolutivos fundamentales, como la recombinación y la variación estructurada.

Esta línea de investigación culmina en 1975 con la publicación de “Adaptation in Natural and Artificial Systems” [32], considerada la obra fundacional de los algoritmos genéticos y punto de partida para su desarrollo moderno. Décadas más tarde, este mismo marco inspiraría el surgimiento de otros métodos de optimización basados en fenómenos naturales, como los algoritmos de enjambre de partículas (PSO) [33] o los algoritmos de búsqueda gravitacional [34], desarrollados ya a finales del siglo XX y principios del XXI.

Los algoritmos genéticos [36] constituyen una subclase específica de los algoritmos evolutivos [35], inspirados en la genética y la selección natural, cuyo objetivo es resolver problemas de optimización complejos mediante un proceso iterativo de mejora de soluciones. Los algoritmos evolutivos abarcan un marco amplio de técnicas bioinspiradas, que incluyen varias subclases como: algoritmos genéticos (AGs), estrategias evolutivas (ES) [39], programación genética (GP) [37] y evolución diferencial (DE) [38]. Entre estas, los algoritmos genéticos implementan de manera más estricta conceptos genéticos, operando sobre poblaciones de soluciones codificadas como cromosomas y aplicando operadores específicos de selección, recombinación y mutación.

Para entender cómo funcionan los algoritmos genéticos, es necesario desglosar sus componentes principales, que determinan el comportamiento de la población y la evolución de las soluciones. Estos incluyen la representación de los individuos, la población inicial, la función de fitness, los operadores genéticos (selección, cruce y mutación) y el ciclo evolutivo que guía la generación de nuevas soluciones. Cada uno de estos elementos desempeña un papel crucial en el equilibrio entre exploración y explotación del espacio de soluciones, asegurando que el algoritmo converja hacia soluciones de alta calidad.

Para entender cómo funcionan los algoritmos genéticos, es necesario desglosar sus componentes principales. La población se refiere al conjunto de individuos en el algoritmo. Cada individuo representa una posible solución al problema, codificada como un cromosoma y cada uno de los valores del cromosoma se llama alelo. La interacción entre estos individuos determina la evolución de las soluciones a lo largo de las generaciones. Para poder representar los cromosomas, se pueden emplear estructuras de datos (como matrices o vectores) utilizando diferentes formatos, como:

- **Representación binaria:** el cromosoma únicamente puede tener dos valores posibles, uno o cero. Resulta útil cuando se quiere implementar algún tipo de lógica booleana (es utilizada en el FARC-HD).
- **Representación entera:** los valores del cromosoma son números enteros.
- **Representación real:** los valores del cromosoma son números reales (es utilizada en el FARC-HD).
- **Representación por permutación:** los valores del cromosoma son números enteros, pero a diferencia del anterior estos no se pueden repetir. Resulta útil cuando se quiere guardar cierto orden.

La función de fitness evalúa la calidad de cada individuo dentro de la población, determinando qué tan adecuada es cada solución respecto a un problema. Esta evaluación guía la selección de los individuos que participarán en la generación siguiente, favoreciendo la reproducción de los que presentan mejores valores de fitness, ya sea seleccionando los de valor máximo si se busca maximización, o los de valor mínimo si se busca minimización. La función de fitness es específica de cada problema y resulta esencial para que el algoritmo genético evolucione correctamente hacia soluciones óptimas.

La selección de individuos [40] es el proceso mediante el cual se eligen los individuos que participarán en la generación de descendencia, en función de los valores obtenidos en la función de fitness. Su objetivo es favorecer a los individuos con mejor desempeño, asegurando que las características más aptas se transmitan a la siguiente generación. Entre los métodos más comunes se encuentran:

- **Selección por ruleta:** Este método asigna a cada individuo de la población una porción de la ruleta proporcional a su valor de fitness, de manera que la suma de todas las porciones sea igual a uno. Los individuos con mejor fitness aumentan sus probabilidades de ser seleccionados para reproducirse. Al utilizar este tipo de selección, la función de fitness se diseña para maximizarse y tomar valores positivos para que cada individuo pueda ser elegido. La selección se realiza generando un número aleatorio entre cero y uno, y eligiendo el individuo cuya sección acumulada contiene ese valor [40].
- **Selección por torneo:** En este método, se elige al azar un grupo de individuos y el de mayor fitness dentro de ese grupo es seleccionado para reproducirse. Torneos con mayor número de individuos aumentan la convergencia, pero pueden reducir diversidad de la población. Para evitar esto se emplea el torneo sin reemplazo, donde cada individuo solo participa una vez por generación, afectando la dinámica de selección y la diversidad de la población [40].
- **Selección por ranking:** en este modelo, los individuos se ordenan según su fitness y se asigna la probabilidad de selección de manera lineal o exponencial, según la posición en la que han sido ordenados. Este método reduce problemas de dominancia de individuos muy aptos y permite un control más estable de la presión selectiva a lo largo de las generaciones [40].
- **Selección elitista:** A diferencia de los métodos anteriores, en la selección elitista no se realiza ninguna comparación entre individuos. En su lugar, se seleccionan los individuos con mejor fitness, asegurando que las soluciones más aptas de la población actual se mantengan intactas en la generación siguiente. Este enfoque garantiza la preservación de las mejores soluciones, pero reduciendo la diversidad.

Para incrementar la diversidad genética en la población, se emplea el cruce [41] entre individuos. Este operador genético combina información de dos o más padres para generar descendencia, permitiendo la exploración de nuevas soluciones dentro del espacio de búsqueda. Dependiendo de la representación de los cromosomas, existen diferentes tipos de cruce:

- **Cruzamiento en un punto:** utilizado en cromosomas con codificación binaria, entera o real. Se selecciona un punto de corte a lo largo de los cromosomas de los padres. Todos los genes antes de este punto se toman de un padre y los genes restantes del otro, generando nuevos hijos [41].
- **Cruzamiento en x puntos:** utilizado en cromosomas con codificación binaria, entera o real. Generalización del cruce anterior, pero en vez de con un punto, se eligen “x” puntos de corte. Esto permite mezclar bloques más grandes de información genética y mantener combinaciones útiles de genes [41].

- **Cruzamiento uniforme:** utilizado en cromosomas con codificación binaria, entera o real. Cada gen del hijo se selecciona de manera independiente de uno de los padres, siguiendo un patrón aleatorio o máscara predefinida. Esta técnica maximiza la diversidad y reduce la dependencia de la posición de los genes en el cromosoma [41].
- **Cruzamiento aritmético:** exclusivo de cromosomas con codificación real. Consiste en calcular combinaciones lineales de los genes de los padres. Tiene la desventaja de que se pierde información de los progenitores. No es obligatorio cruzar todos los valores.
- **Cruzamiento parcialmente mapeado (PMX):** exclusivo de cromosomas con codificación por permutación. Intercambia un segmento de genes entre dos padres y luego reubica los genes restantes según un mapa de correspondencia, asegurando que no haya duplicados en el hijo.
- **Cruzamiento por aristas:** exclusivo de cromosomas con codificación por permutación. Construye al hijo seleccionando sucesivamente nodos que mantengan la mayor cantidad de aristas existentes en los padres, útil para rutas o secuencias.

Otra manera que existe para incrementar la diversidad genética en la población consiste en emplear la mutación [41], modificar alguno de los valores de un cromosoma sin depender de información que tenga el resto. La probabilidad de mutación generalmente es menor que la probabilidad de cruzamiento. Dependiendo de la representación de los cromosomas, existen diferentes tipos de mutaciones:

- **Mutación de valor binario:** exclusiva de cromosomas con codificación binaria. Consiste en invertir uno o varios genes de un cromosoma.
- **Mutación por reinicio aleatorio:** utilizada en cromosomas con codificación entera. El valor del gen se reemplaza por otro posible valor aleatorio dentro de su rango.
- **Mutación por deslizamiento:** empleada en cromosomas con codificación entera. Se suma o resta un pequeño valor al gen mutado, siguiendo una distribución normal centrada en cero.
- **Mutación uniforme:** utilizada en cromosomas con codificación real. Cada gen tiene probabilidad de ser reemplazado por un valor aleatorio dentro de su rango permitido.

- **Mutación no uniforme según una distribución fija:** exclusiva de cromosomas con codificación real. Se ajusta el gen mutado siguiendo una distribución predefinida, similar a la mutación por deslizamiento.
- **Mutación por intercambio:** utilizada en cromosomas con codificación por permutación. Se intercambian las posiciones de dos genes seleccionados aleatoriamente.
- **Mutación por inserción:** utilizada en cromosomas con codificación. Un gen seleccionado se inserta en otra posición del cromosoma.
- **Mutación por sacudida:** utilizada en cromosomas con codificación. En este caso, se reordena un segmento de genes de manera aleatoria para aumentar diversidad.
- **Mutación por inversión:** empleado en cromosomas con codificación por permutación. Un segmento de genes del cromosoma se invierte, manteniendo la validez del cromosoma.

El ciclo de un algoritmo genético (AG) se ilustra en la Figura 9. Este ciclo comienza con la inicialización de una población de cromosomas y continúa mediante la evaluación de su aptitud. Posteriormente, se aplica la selección de individuos que participarán en la generación de descendencia, seguida de los operadores de cruce y mutación para producir nuevos individuos. La fase de reemplazo determina cómo la descendencia sustituye a la población actual, y finalmente, el algoritmo evalúa si se cumple el criterio de terminación, ya sea por alcanzar un número máximo de generaciones o por encontrar una solución óptima. Este proceso se repite de manera iterativa, permitiendo que la población evolucione progresivamente hacia soluciones de mayor calidad (ver Figura 9).

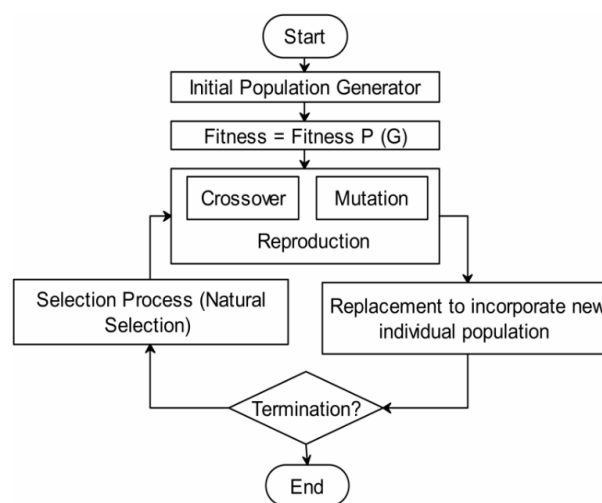


Figura 9: ciclo de un algoritmo genético [35]

Los algoritmos genéticos destacan por su robustez, capacidad para explorar grandes espacios de búsqueda e interpretabilidad, manteniendo diversidad de soluciones gracias a su enfoque poblacional. No obstante, por sí solos pueden estancarse en óptimos locales, por lo que su rendimiento depende del ajuste de parámetros y de la naturaleza del problema. Según la teoría del No Free Lunch Theorem [42], no existe un algoritmo de optimización universalmente superior; su eficacia es específica de cada problema. Por ello, los algoritmos evolutivos pueden beneficiarse de estrategias paralelas o híbridas, como los algoritmos meméticos, que combinan exploración global con optimización local, mejorando eficiencia y convergencia hacia soluciones óptimas.

3. Algoritmo FARC-HD

3.1. Introducción

El algoritmo FARC-HD (Fuzzy Association Rule-based Classification for High-Dimensional Problems) [4] pertenece a la familia de los SCBRD, puesto que representan el conocimiento mediante reglas lingüísticas del tipo “Si A entonces C ”, utilizando etiquetas difusas para modelar fenómenos con incertidumbre y favorecer la interpretabilidad. En concreto, las reglas en el FARC-HD tienen un formato:

Regla R_j : Si x_1 es A_{j1} y x_2 es A_{j2} y ... x_m es A_{jm} entonces Clase = C_j con RW_j ;

Donde:

- **R_j** : identifica de manera única la regla j dentro del conjunto de reglas.
- **X_i** : representa la i -ésima variable de entrada.
- **A_{ji}** : es la etiqueta lingüística difusa asociada a la variable X_i .
- **C_j** : es la clase de salida.
- **RW_j** : es el peso asignado a la regla, normalmente calculado mediante WR_{Acc} .

Los SCBRD han sido ampliamente utilizados en tareas de clasificación por su capacidad para integrar conocimiento comprensible por humanos y ofrecer modelos transparentes. Sin embargo, FARC-HD introduce avances clave que lo diferencian de los SCBRD tradicionales. En primer lugar, mientras que los SCBRD convencionales suelen trabajar con un número moderado de atributos y basan la construcción de reglas en heurísticas simples o en particiones fijas del espacio difuso, FARC-HD fue diseñado específicamente para entornos de alta dimensionalidad, donde el número de posibles combinaciones difusas crece exponencialmente y la generación de reglas se vuelve computacionalmente costosa. Por ello, el algoritmo incorpora un enfoque híbrido que combina:

- **Extracción de reglas de asociación difusas**, que permite generar reglas iniciales altamente informativas sin explorar de manera exhaustiva el espacio de búsqueda.
- **Técnicas evolutivas de selección y ajuste**, que optimizan la base de reglas para reducir redundancias, mejorar la precisión y ajustar lateralmente las etiquetas lingüísticas.

Este enfoque fue formalizado por Alcalá-Fdez, Alcalá y Herrera en [4], donde los autores proponen un modelo capaz de mantener interpretabilidad y rendimiento competitivo incluso cuando se trabaja con decenas o cientos de atributos, algo en lo que

el resto de SCBRD tienden a sufrir tanto por la explosión combinatoria como por la dificultad de ajustar los parámetros difusos.

3.2. Fases del algoritmo

FARC-HD se estructura en tres fases principales, tal y como se presenta en la figura 10 y como se describe en su propuesta original [4]:

1. **Extracción de reglas de asociación difusas:** Se generan reglas mediante un árbol de búsqueda basado en una adaptación difusa del algoritmo Apriori, lo que permite enumerar todos los ítemsets frecuentes en términos difusos.
2. **Preselección de reglas candidatas:** Cada regla generada es evaluada mediante criterios de calidad, con el objetivo de filtrar aquellas que no presentan valor discriminativo o que podrían introducir ruido en el modelo final.
3. **Selección genética y ajuste lateral:** Las reglas supervivientes al filtrado previo se someten a un proceso evolutivo que combina selección genética y ajuste lateral. Esta etapa optimiza el conjunto final de reglas del clasificador.

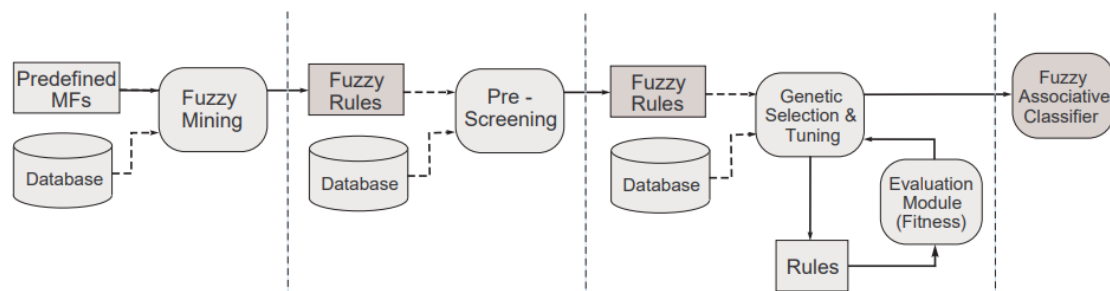


Figura 10: Fases principales del FARC-HD [4]

Adicionalmente, el modelo incorpora una regla por defecto que asigna la clase mayoritaria del conjunto de entrenamiento, garantizando la cobertura de instancias no descritas por ninguna regla activa.

3.2.1 Extracción de reglas de asociación difusas de clasificación

La primera fase del algoritmo FARC-HD se centra en la extracción de reglas de asociación difusas orientadas a la clasificación, cuyo propósito es generar un conjunto inicial de reglas candidatas a partir de los ítemsets difusos frecuentes correspondientes a cada clase. Para ello, el método emplea un esquema de búsqueda sistemático basado en un árbol de ítemsets, cuya estrategia de expansión y poda se inspira directamente en los principios del algoritmo Apriori, propuesto originalmente por R. Agrawal y R. Srikant en 1994 [43].

Al igual que Apriori, esta fase sigue una exploración incremental desde los 1-ítemsets hasta los k-ítemsets, aplica el principio de clausura descendente del soporte y descarta

cualquier candidato cuyos subconjuntos no sean frecuentes. Sin embargo, FARC-HD introduce algunas variaciones nuevas:

- Uso de soporte difuso en lugar de soporte tradicional
- Generación de reglas específicas por clase, orientadas a la tarea de clasificación y no a la asociación general
- Sustitución de la pertenencia booleana por grados de membresía obtenidos a partir de funciones difusas
- Cálculo de coincidencia mediante T-normas
- Incorporación de una poda basada en la confianza máxima para reducir el número final de reglas candidatas.

El proceso parte de un nodo raíz vacío y genera sucesivos niveles con los ítemsets de tamaño creciente:

- **Nivel 1:** incluye los ítemsets de un solo elemento, correspondientes a los términos lingüísticos de cada atributo. Si un atributo posee q_j términos lingüísticos, generará q_j ítemsets en este primer nivel.
- **Niveles superiores:** los hijos de un nodo representan ítemsets ampliados con atributos posteriores según el orden de aparición en los datos y evitando duplicidades.

Cuando un atributo tiene más de dos términos ($q_j > 2$), se reemplaza por q_j variables binarias, garantizando que no aparezcan simultáneamente varios términos del mismo atributo en un único ítemset (tal como se muestra en la figura 11).

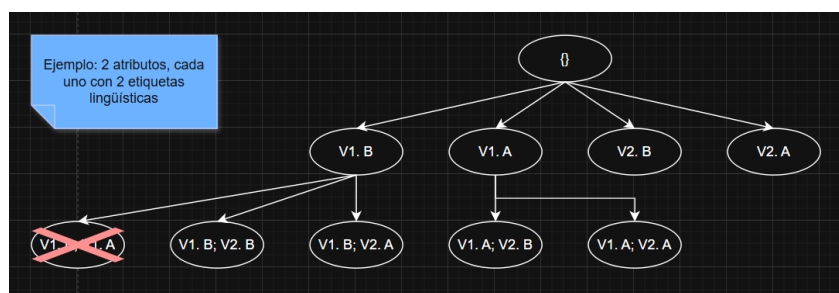


Figura 11: generación de ítemsets

El soporte de un ítemset se calcula mediante el soporte difuso y se considera frecuente si supera un soporte mínimo previamente establecido. A diferencia de Apriori, que utiliza un único soporte mínimo global, FARC-HD ajusta el soporte mínimo para cada clase según su distribución en el conjunto de entrenamiento. Esta adaptación permite

detectar patrones relevantes en clases minoritarias y evita que queden poco representadas durante la generación de reglas.

Además, se tiene en cuenta la confianza de cada nodo (también explicada previamente). Si la regla generada por un ítemset alcanza un umbral máximo de confianza establecido por el usuario, no se generan más nodos a partir de ese ítemset, dado que se considera que la regla ya cumple con la calidad requerida.

Una vez obtenidos todos los ítemsets frecuentes de cada clase, se generan las reglas de asociación difusas para clasificación. Para cada ítemset frecuente A perteneciente a la clase C_j , donde A está compuesto por todos los ítems (etiquetas lingüísticas) del ítemset que genera la regla, se crea una regla del tipo:

Si A entonces C_j

Este proceso se repite para cada clase presente en el conjunto de entrenamiento, conformando así un conjunto inicial y exhaustivo de reglas candidatas que serán posteriormente refinadas en las fases sucesivas del algoritmo FARC-HD.

3.2.2 Preselección de reglas candidatas

Tras la generación masiva de reglas candidatas en la fase anterior, esta etapa tiene como objetivo preseleccionar las reglas más relevantes, reduciendo así los costos computacionales de la siguiente fase. Esto es especialmente importante, ya que FARC-HD suele aplicarse a conjuntos de datos muy grandes que generan un número elevado de reglas. Para lograrlo, se emplea un enfoque basado un esquema de ponderación de patrones [44].

En este esquema, los patrones positivos cubiertos por una regla seleccionada no se eliminan inmediatamente, sino que cada patrón mantiene un contador i que indica cuántas veces ha sido cubierto por las reglas seleccionadas. Los pesos de los patrones cubiertos disminuyen siguiendo la fórmula:

$$w(e_j, i) = \frac{1}{i + 1}$$

En la primera iteración ($i = 0$), todos los patrones de la clase objetivo tienen peso 1. A medida que se seleccionan reglas, los patrones ya cubiertos ven reducidos sus pesos, mientras que los patrones no cubiertos mantienen su peso inicial, aumentando sus posibilidades de ser seleccionados en iteraciones posteriores. Finalmente, los patrones se eliminan por completo cuando han sido cubiertos más de k_t veces.

El procedimiento iterativo del algoritmo se aplica para cada clase y sigue los siguientes pasos:

- Se ordenan las reglas según un criterio de evaluación, de mejor a peor. Este criterio es la evaluación mediante la formula:

$$wWRAcc''(A \Rightarrow C_j) = \frac{n''(A \cap C_j)}{n_0(C_j)} \left(\frac{n''(A \cap C_j)}{n''(A)} - \frac{n(C_j)}{N} \right)$$

Donde:

- $n''(A)$ representa la suma de los productos de los pesos y los grados de coincidencia de los patrones cubiertos con el antecedente de la regla.
- $n''(A \cap C_j)$ es la suma de los productos de los pesos y los grados de coincidencia de los patrones correctamente cubiertos.
- $n_0(C_j)$ es la suma de los pesos de los patrones de la clase C_j .
- Se selecciona la mejor regla y se vuelven a ponderar los patrones cubiertos por la regla

El proceso se repite hasta que se cumpla alguna de las condiciones de parada: todos los patrones hayan sido cubiertos más de k_t veces o no queden más reglas en la base de reglas. Las reglas seleccionadas tendrán valores en el intervalo $[-1,1]$, donde valores cercanos a 1 indican reglas especialmente útiles para la clasificación.

3.2.3 Selección genética de reglas y ajuste lateral de las funciones de pertenencia

En esta última fase se emplean algoritmos genéticos [32] para seleccionar un conjunto de reglas con alta precisión de clasificación a partir de la base de reglas obtenida en la fase anterior. Para ello, se considera el enfoque propuesto en [35], en el que las reglas se representan mediante la representación lingüística 2-tuplas [55]. Con esta representación, cada etiqueta lingüística se define mediante dos valores: la etiqueta original y un parámetro de desplazamiento lateral, denotado como β comprendido en el intervalo $[-0.5, 0.5]$.

Este parámetro β permite ajustar la posición de la función de pertenencia para mejorar la cobertura de los patrones sin modificar su forma, manteniendo así la interpretabilidad de las etiquetas. El ajuste lateral se aplica de forma global, de manera que el mismo valor de β se utiliza en todas las reglas donde aparece dicha etiqueta. Esta estrategia reduce significativamente la complejidad del espacio de búsqueda y permite representar el sistema difuso basado en 2-tuplas como un FRBCS clásico tipo Mamdani, sin alterar la clase de salida ni el peso asignado a las reglas.

Para la selección de reglas y el ajuste lateral se utiliza el algoritmo genético CHC [56], elegido por su buen equilibrio entre exploración y explotación, adecuado para problemas con espacios de búsqueda complejos. Este modelo genético cuenta con las siguientes consideraciones:

- **Selección de población:** los padres y su descendencia compiten por integrar la siguiente generación
- **Prevención de incesto:** determina si dos padres pueden cruzarse según su distancia Hamming respecto a un umbral L . Este umbral se decrementa progresivamente (en una unidad si no se generan individuos nuevos, o en un 10 % de su valor original si los hay) y provoca el reinicio de la población cuando alcanza valores negativos.
- **Codificación de cromosomas:** se emplea un esquema doble, representando cada cromosoma como $C = CS \parallel CT$:
 - **CS:** codifica la selección de reglas mediante un vector binario, donde '1' indica que la regla está seleccionada y '0' lo contrario
 - **CT:** codifica los parámetros β de cada etiqueta mediante números reales.
- **Población inicial:** se incluye un cromosoma con todas las reglas seleccionadas (valores '1' en CS) y sin desplazamiento lateral (valores '0.0' en CT), mientras que el resto de los individuos se generan aleatoriamente.
- **Evaluación de cromosomas:** se realiza mediante una función de fitness que combina precisión de clasificación y complejidad del modelo, penalizando cromosomas con clases sin reglas o patrones no cubiertos. La función se define como:

$$Fitness(C) = \frac{\#hits}{N} - \delta * \frac{NR_{inicial}}{NR_{inicial} - NR + 1.0}$$

Donde:

- **#Hits:** número de ejemplos o patrones correctamente clasificados por el conjunto de reglas seleccionado en el cromosoma C .
- **N :** número total de ejemplos del conjunto de entrenamiento.
- **δ :** porcentaje de ponderación definido por el usuario.
- **$NR_{inicial}$:** número total de reglas candidatas al inicio.
- **NR :** número de reglas seleccionadas por el cromosoma C .

- El operador de cruce depende de la parte del cromosoma:
 - **PCBLX:** empleado en CT. Utiliza en concepto de vecindad, generando descendientes alrededor de los genes de los padres o en un intervalo determinado por ambos padres.
 - **HUX:** empleado en CS. Se intercambia la mitad de los alelos diferentes entre los padres, garantizando máxima exploración.

En cada generación se combinan los descendientes de ambas partes, seleccionándose los mejores para la siguiente generación. Para calcular la distancia Hamming en CT se transforma cada gen a código Gray con un número fijo de bits.

- **Reinicio:** A diferencia de otros algoritmos genéticos, CHC no cuenta con un proceso de mutación genética para no caer en óptimos locales. En vez de eso, cuenta con un procedimiento de reinicio se utiliza cuando el umbral L es menor que cero, indicando que los individuos de la población son muy similares. Se mantiene el mejor cromosoma y se generan aleatoriamente tantos cromosomas para igualar la población inicial.

4. Entorno de experimentación

4.1 Entorno de desarrollo

El desarrollo de los experimentos se ha realizado utilizando Python, un lenguaje de programación ampliamente adoptado en los campos de análisis de datos, aprendizaje automático e inteligencia artificial. Aunque no es el lenguaje más rápido en términos de ejecución pura, su popularidad en IA se explica por la gran cantidad de librerías especializadas y optimizadas que simplifican el desarrollo de modelos avanzados sin necesidad de programar cada detalle desde cero.

Las librerías que han sido utilizadas en el proyecto son:

- **NumPy:** proporciona estructuras de datos como arrays y matrices multidimensionales, y funciones matemáticas altamente optimizadas para cálculos numéricos.
- **Numba:** permite la compilación just-in-time (JIT) de funciones Python, acelerando significativamente bucles y operaciones matemáticas críticas, especialmente en algoritmos iterativos [17].
- **time y cProfile:** para la medición precisa del tiempo de ejecución y la monitorización del rendimiento del código.
- **sys y traceback:** facilitan la gestión de errores y excepciones, mejorando la robustez de los experimentos.
- **Collections:** se utiliza para estructuras de datos que mantienen el orden de inserción, útil en el manejo de reglas y resultados.
- **re:** permite operaciones avanzadas de búsqueda y manipulación de cadenas mediante expresiones regulares, importante para el preprocesamiento de datos.
- **os:** proporciona acceso al sistema de archivos y otras funcionalidades del entorno operativo, necesarias para la carga y gestión de conjuntos de datos.

Estas librerías, combinadas con la capacidad de Python para integrar optimizaciones y estructuras de alto nivel, permiten un desarrollo rápido y eficiente de experimentos en inteligencia artificial y minería de datos.

4.2 Conjunto de datos

Para la evaluación del algoritmo se han utilizado 21 conjuntos de datos procedentes del repositorio KEEL [48], reconocidos en la literatura por su diversidad y utilidad en problemas de clasificación supervisada. Estos datasets presentan distintas características en cuanto al número de atributos, clases y tamaño de los ejemplos, lo

que permite realizar un análisis exhaustivo del desempeño del modelo bajo diferentes condiciones.

Para facilitar la observación de patrones de comportamiento del algoritmo, se han identificado tres subconjuntos: ocho datasets comparten el mismo número de clases, siete tienen un número de atributos parecido y seis presentan un número muy similar de ejemplos. La tabla 1 resume las principales características de cada conjunto de datos, destacando los tres subconjuntos mediante colores:

Dataset	Nº de Atributos (R/I/N)	Nº de Clases	Nº de Ejemplos
Wine	13 (13/0/0)	3	178
Balance	4 (4/0/0)	3	625
Contraceptive	9 (0/9/0)	3	1473
Newthyroid	5 (4/1/0)	3	215
thyroid	21 (6/15/0)	3	7200
Post-operative	8 (0/0/8)	3	87
Splice	60 (0/0/60)	3	3190
Iris	4 (4/0/0)	3	150
Glass	9 (9/0/0)	7	214
Tic-tac-toe	9 (0/9/0)	2	958
Wisconsin	9 (0/9/0)	2	683
Breast	9 (0/0/9)	2	277
Shuttle	9 (0/9/0)	7	58000
Saheart	9 (5/3/1)	2	462
magic	10 (10/0/0)	2	19020
Banana	2 (2/0/0)	2	5300
Phoneme	5 (0/5/0)	2	5404
Page-blocks	10 (4/6/0)	5	5472
texture	40 (40/0/0)	11	5500
Mushroom	22 (0/0/22)	2	5644
Satimage	36 (0/36/0)	7	6435

Tabla 1: datasets empleados en el trabajo y sus propiedades

Para dividir los ejemplos de cada conjunto de datos, la propia plataforma KEEL proporciona una partición cross-validation [46] de 5 bloques. Esto permite, para cada partición, utilizar 4/5 de los datos como conjunto de entrenamiento y el 1/5 restante como conjunto de prueba. Sin embargo, dado que en los experimentos el objetivo principal es evaluar el tiempo de ejecución del algoritmo, y no su eficacia (pues el sistema está diseñado para que la implementación en Java obtenga la misma accuracy que la versión en Python), se ha optado por utilizar únicamente la primera partición generada por KEEL. Esta decisión reduce significativamente el tiempo necesario para obtener resultados, aspecto especialmente relevante en los experimentos iniciales, donde los tiempos de ejecución son considerablemente más elevados.

5. Optimización y análisis de rendimiento

Previo a la ejecución del programa en Python en sus diferentes versiones, resulta conveniente analizar los resultados obtenidos con la implementación original en Java. Estos datos sirven como referencia y permiten evaluar de manera objetiva las mejoras en tiempo de ejecución logradas durante el proceso de optimización. La Tabla 2 recoge los tiempos correspondientes a las tres fases principales del sistema: extracción de reglas mediante el algoritmo A Priori (TA), optimización mediante el algoritmo genético (TG) y tiempo total de ejecución (TT) para cada conjunto de datos.

Dataset	TA	TG	TT
Wine	00:00:12	00:00:51	00:01:03
Balance	00:00:07	00:05:20	00:05:28
Contraceptive	00:00:50	00:29:12	00:30:03
Newthyroid	00:00:02	00:00:43	00:00:46
thyroid	00:01:09	00:20:30	00:21:01
Post-operative	00:00:04	00:03:04	00:03:55
Splice	11:05:57	04:03:22	15:08:40
Iris	00:00:02	00:00:23	00:00:25
Glass	00:00:08	00:02:18	00:02:27
Tic-tac-toe	00:00:49	00:06:39	00:06:50
Wisconsin	00:00:05	00:01:57	00:02:03
Breast	00:00:10	00:00:58	00:01:08
Shuttle	00:05:12	02:22:20	02:27:12
Saheart	00:00:17	00:05:47	00:06:06
magic	00:06:21	03:55:39	04:01:27
Banana	00:00:06	00:11:25	00:11:34
Phoneme	00:00:25	00:23:26	00:23:13
Page-blocks	00:00:39	00:26:24	00:27:06
texture	01:27:35	01:59:33	03:27:12
Mushroom	00:03:42	00:02:02	00:05:46
Satimage	01:41:26	01:17:29	02:58:18

*Tabla 2: resultados algoritmo en versión Java en formato
minutos:segundos:milisegundos*

Para facilitar el análisis global del rendimiento, la Tabla 3 presenta los tiempos medios calculados para cada uno de los tres subgrupos de datasets identificados en la Tabla 2 (diferenciados por color). De esta forma, se puede evaluar el impacto de las características de cada grupo en los tiempos de ejecución de las fases TA, TG y el tiempo total en función del mismo número de clases, parecido número de atributos y parecido número de ejemplos.

Dataset	Media TA	Media TG	Media TT
DS con 3 clases	01:23:32	00:37:55	02:01:25
DS con 9/10 atributos	00:01:51	00:56:31	00:58:10
DS con 5300/6400 ejemplos	00:32:18	00:43:23	01:15:31

Tabla 3: Resumen de medias de los tiempos de ejecución en Java por grupos de datasets en formato minutos:segundos:milisegundos

A partir de estos resultados pueden extraerse conclusiones relevantes para las futuras implementaciones en Python. Como puede apreciarse en la Figura 11, existe una correlación claramente positiva entre el número de atributos de cada dataset y el tiempo total de ejecución, mientras que dicha correlación es considerablemente menor respecto al número de clases o al número de ejemplos. Esta tendencia también se refleja en el tiempo necesario para ejecutar el A Priori, cuyo comportamiento depende casi exclusivamente del número de atributos. Por el contrario, el tiempo de ejecución del algoritmo genético muestra una relación alta tanto con el número de atributos como con el número de ejemplos.

Esto es lógico debido a que la evaluación del fitness de cada individuo requiere comprobar el desempeño de sus reglas sobre todo el conjunto de datos, lo que implica recorrer cada ejemplo y procesar todos los atributos relevantes. Como este proceso se repite en cada generación y para todos los individuos de la población, el coste computacional crece linealmente con el número de patrones y con el número de atributos, haciendo al algoritmo genético especialmente sensible a datasets grandes y de alta dimensionalidad.

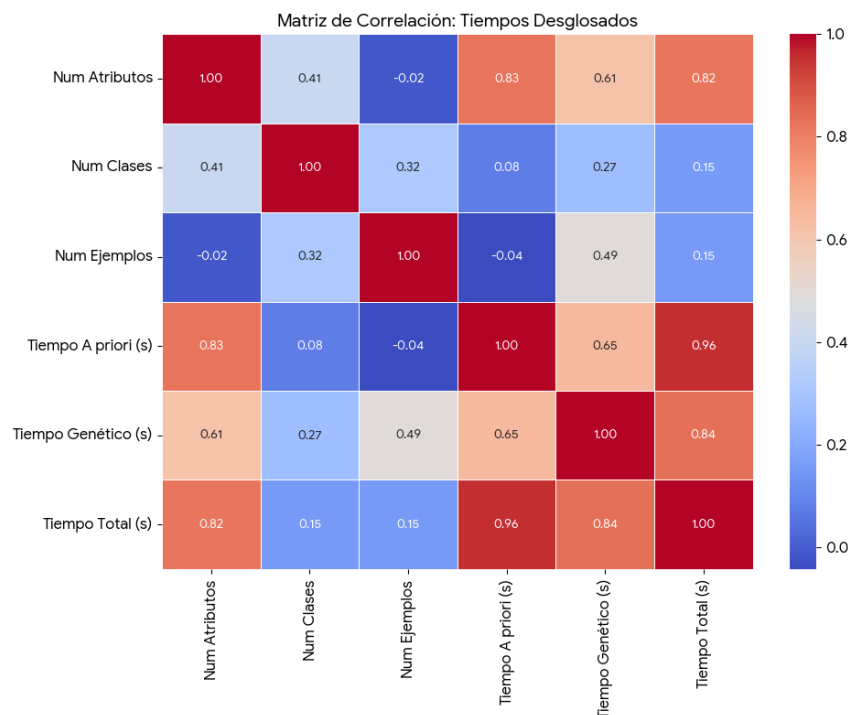


Figura 11: matriz de correlaciones de los datasets

5.1 Implementación base (versión pura en Python)

Esta versión constituye la primera aproximación del algoritmo FARC-HD al entorno Python, siendo la más básica de todas. No obstante, ha sido también la que ha requerido

un mayor tiempo de desarrollo, ya que a partir de ella se construyen y optimizan las versiones posteriores. Para esta implementación se realizó una traducción directa del código original en Java, manteniendo la misma estructura lógica y el mismo flujo de ejecución y sin introducir cambios orientados al rendimiento ni aprovechar las particularidades del lenguaje Python. La prioridad en esta etapa ha sido asegurar la correcta equivalencia funcional con el código Java.

La única librería externa utilizada desde el inicio ha sido NumPy, pero con un uso muy limitado. NumPy se ha incorporado para facilitar el manejo de los vectores de datos procedentes de los conjuntos de entrenamiento y prueba, dado que permite almacenarlos de forma más compacta y acceder a ellos con mayor comodidad. Sin embargo, en esta versión no se aplican operaciones vectorizadas y los arrays se recorren mediante bucles explícitos, los cuales son más ineficientes que los de Java debido a las diferencias entre ambos lenguajes: Java ejecuta los bucles sobre la JVM utilizando un compilador JIT que optimiza dinámicamente el código, mientras que Python los evalúa de forma interpretada, instrucción por instrucción.

A continuación, en la Tabla 4 se presentan los resultados obtenidos con la versión base en Python, siguiendo el mismo formato empleado en la implementación original en Java. Con el fin de evitar tiempos de ejecución desproporcionados, se ha establecido un criterio de parada basado en la comparación con la versión Java: cuando el tiempo de ejecución de alguna de las fases del algoritmo (Apriori o Genético) supera ampliamente el tiempo correspondiente en Java, la ejecución puede ser detenida y el resultado se marca como excesivo. Pese a que en un primer momento se ha valorado cancelar automáticamente cualquier ejecución que superase 50 veces el tiempo original en cualquiera de las dos partes, únicamente un número muy reducido de conjuntos de datos habría finalizado su ejecución. Por este motivo, en este primer análisis comparativo se optó por relajar dicha restricción, permitiendo completar aquellas ejecuciones que, aun siendo varias decenas de veces más lentas que la versión en Java, finalizaban en un tiempo razonable. La cancelación se realizó únicamente en los casos en los que el tiempo de ejecución resultó claramente inmanejable.

En los casos marcados como excesivo no se dispone de un valor temporal real; sin embargo, se considera que su comportamiento es suficientemente ineficiente como para resultar poco útil en la práctica. Este criterio permite centrar el análisis en los casos más representativos y evitar tiempos de espera excesivos. Es previsible que, a medida que se introduzcan mejoras en versiones posteriores, el número de resultados reales obtenidos aumente.

La relación de tiempos de ejecución entre la versión de Python y la de Java se muestra tras cada columna temporal mediante el factor x_{TX} , que indica cuántas veces el tiempo de ejecución en Python supera o es inferior al correspondiente en Java.

dataset	TA	xTA	TG	xTG	TT	xTT
wine	00:00:19:30	167	00:01:42:89	203	00:02:02:19	195
balance	00:00:04:06	57	00:22:56:79	286	00:23:00:86	282
contraceptive	excesivo	----	excesivo	----	excesivo	----
newthyroid	00:00:00:41	18	00:00:01:24	195	00:01:24:41	184
thyroid	00:03:20:82	290	01:16:35:17	226	01:19:55:99	228
post-operative	00:00:05:47	124	00:00:30:38	165	00:00:35:85	152
splice	excesivo	----	excesivo	----	excesivo	----
iris	00:00:00:14	8	00:00:20:52	90	00:00:20:66	81
glass	00:00:14:98	180	00:07:46:35	261	00:08:01:33	257
tic-tac-toe	00:04:03:15	492	excesivo	----	excesivo	----
wisconsin	00:00:02:77	55	00:08:57:90	340	00:09:00:68	329
breast	00:00:27:05	260	00:02:44:07	283	00:03:11:75	273
shuttle	excesivo	----	excesivo	----	excesivo	----
saheart	00:00:27:99	157	00:26:26:23	289	00:26:54:23	285
magic	00:30:38:68	316	excesivo	----	excesivo	----
banana	00:00:01:86	32	00:59:00:53	326	00:59:02:40	323
phoneme	00:00:33:87	136	02:11:22:82	344	02:11:56:70	342
page-blocks	00:02:05:34	321	excesivo	----	excesivo	----
texture	excesivo	----	excesivo	----	excesivo	----
mushroom	00:35:16:72	618	excesivo	----	excesivo	----
satimage	excesivo	----	excesivo	----	excesivo	----

Tabla 4: datos de la versión base en Python en formato horas:minutos:segundos:milisegundos para las mediciones temporales

La Tabla 5 resume el rendimiento de la implementación en Python. A diferencia de la utilizada en la versión en Java, los datasets que presentaron tiempos de ejecución excesivos han sido excluidos del cálculo de la media para no distorsionar los resultados. Las columnas muestran:

- Tipo de datasets
- Métrica recogida (TA, TG, o TT)
- Tiempo medio de cada métrica (formato HH: MM: SS)
- Tiempo de ejecución relativo respecto a java
- Datasets válidos para calcular las medias

Datasets	Métrica	Tiempo medio (HH: MM: SS)	veces java	Datasets validos
DS con 3 clases	TA	0:00:38	110	6 de 8
	TG	0:17:01	194	6 de 8
	TT	0:17:53	187	6 de 8
DS con 9/10 atributos	TA	0:05:59	243	6 de 7
	TG	00:11:28	293	4 de 7
	TT	00:11:46	286	4 de 7
DS con 5300/6400 ejemplos	TA	00:09:29	276	4 de 6
	TG	01:35:11	335	2 de 6
	TT	01:35:29	332	2 de 6

Tabla 5: Resumen de los tiempos de ejecución en Python por grupos de datasets

Tras obtener los resultados de la primera versión en Python y contrastarlos con la versión original en Java, se confirma la hipótesis esperada: la implementación es ineficiente tanto en la fase de extracción de reglas (A Priori) como en la etapa evolutiva (Genético).

A continuación, se presenta la matriz de correlaciones (Figura 12), calculada exclusivamente a partir de los conjuntos de datos que lograron finalizar su ejecución. Aunque esta matriz no incluye la totalidad de los casos, ofrece una visión general representativa de las dependencias temporales.



Figura 12: matriz de correlaciones de la versión 1 de Python.

En líneas generales, el análisis de los tiempos de ejecución permite extraer las siguientes observaciones:

- **Sensibilidad a la dimensionalidad en el algoritmo A Priori:** Se observa una correlación directa entre el número de atributos y la reducción del rendimiento en esta fase. Mientras que en datasets con baja dimensionalidad (como Iris o Banana) el algoritmo es entre 8 y 30 veces más lento que su contraparte en Java, esta cifra se dispara drásticamente al aumentar el número de atributos, como ocurre en Mushroom (22 atributos) o Page-blocks (10 atributos). Esto confirma que los bucles explícitos de Python para el cálculo de frecuencias y soportes no gestionan eficazmente espacios de búsqueda amplios, donde es necesario verificar múltiples condiciones simultáneamente.
- **Dependencia crítica del volumen de datos en el Algoritmo Genético:** Existe una relación muy grande entre el número de ejemplos y el tiempo de ejecución de la etapa evolutiva, siendo el principal cuello de botella del sistema. En datasets con un gran volumen de instancias (Shuttle, Magic, Texture), la ejecución se vuelve inviable en tiempos razonables. Incluso en conjuntos más reducidos (Balance, Phoneme), el tiempo de ejecución se mantiene muy elevado. Esto es debido a la evaluación iterativa en funciones como fitness, que obliga a recorrer la totalidad de los ejemplos para cada individuo de la población en cada generación, acumulando una carga de procesamiento masiva.
- **Irrelevancia del número de clases:** Finalmente, los datos sugieren que el número de clases del problema no constituye un factor determinante en la explosión de los tiempos de ejecución.

Con el objetivo de identificar los cuellos de botella específicos a nivel de código, se ha procedido a realizar un análisis de perfilado utilizando la herramienta estándar Profile. Este análisis permite desglosar los tiempos acumulados de las funciones internas de FARC-HD. El dataset seleccionado para este análisis será Wisconsin, debido a que presenta un equilibrio representativo en sus dimensiones y sus tiempos de ejecución en la versión base (aproximadamente 9 minutos) son lo suficientemente extensos pero manejables para capturar datos significativos.

Tras ejecutar el perfilado sobre el conjunto de datos Wisconsin, se desglosaron los tiempos de ejecución por función. La Tabla 6 resume las cinco funciones con mayor consumo de CPU sobre un tiempo total de 773.6 segundos, identificando así los puntos críticos donde se concentra la carga computacional.

Función (Archivo)	N.º de Llamadas	Tiempo (s)	% del T. Total
degreeProduct (Rule.py)	104,138,298	361.65 s	46.7%
matching (DataBase.py)	608,291,881	158.83 s	20.5%
FRM_AC (RuleBase.py)	5,669,255	122.71 s	15.8%
Fuzzifica (Fuzzy.py)	120,631,930	56.06 s	7.2%
matching (Rule.py)	104,138,298	32.04 s	4.14%

Tabla 6: Funciones con mayor coste computacional (Dataset: Wisconsin).

El análisis de estos resultados permite extraer conclusiones sobre la ineficiencia de la implementación base y los lugares donde hay que incidir para reducir, en la medida de lo posible, el tiempo de ejecución.

5.2 Uso de máscaras de datos

Tomando como base el análisis de tiempos globales (21 datasets) y el perfilado detallado (dataset Wisconsin), se ha evidenciado que el coste computacional se dispara debido a la repetición masiva de cálculos idénticos (como la fuzzificación) y a la evaluación de reglas sobre datos que no aportan información (grados de pertenencia nulos). Con el objetivo de mitigar estos cuellos de botella, se han implementado cambios estructurales orientados a sustituir el procesamiento escalar por operaciones matriciales y pre-calculadas. Las dos principales mejoras introducidas en esta versión son:

- **Pre-fuzzificación vectorial de los datos:** El análisis de perfilado reveló que la función Fuzzifica era invocada más de 120 millones de veces. En esta segunda versión, se ha modificado la arquitectura para realizar una fuzzificación más eficiente. Utilizando operaciones vectoriales, se transforma la totalidad del conjunto de datos de entrenamiento en una matriz bidimensional (donde cada columna representa una etiqueta lingüística y cada fila un ejemplo). Gracias a esta estrategia, la función matching deja de realizar cálculos matemáticos y pasa a realizar simples consultas de acceso directo a memoria, reduciendo el coste de CPU prácticamente a cero.
- **Generación de matrices de reglas activas (Máscaras de datos):** se detectó que el algoritmo Genético evalúa un subconjunto de reglas en cada iteración. En esta segunda versión, se implementó el método obtenerLabelsUsadas, que genera dinámicamente una máscara de índices, analiza el cromosoma del individuo actual y extrae únicamente las reglas activas. Esto permite que la fuzzificación actualice los valores en aquellos valores donde van a ser necesarios (afectados por el desplazamiento lateral).
- **Adaptación de la clase Rule para acceso indexado:** Como consecuencia de la pre-fuzzificación, se modificó el método matching en la clase Rule. En la versión

optimizada, este método se ha modificado para realizar un acceso directo a la matriz de datos fuzzificados.

Con todos nuevos datos implementados procedo a calcular la eficiencia de los datasets pero con un pequeño cambio en el orden de realización de los experimentos: primero realizare el análisis con el dataset individual y, si obtengo una mejora significativa, el análisis global con todos los datasets.

Función (archivo)	Nº de Llamadas	Tiempo	% del T. Total
degreeProduct (Rule.py)	104,138,298	334.15 s	50.60%
Matching (DataBase.py)	608,291,831	133.89 s	20.27%
FRM_AC (RuleBase.py)	5,669,255	114.96 s	17.41%
Matching (Rule.py)	104,138,298	25.66 s	3.88%
StringRep (individual.py)	7,400	9.39 s	1.42%

Tabla 7: Funciones con mayor coste computacional (Dataset: Wisconsin) 2.

El tiempo de construcción del FARC HD en el dataset Wisconsin ha sido de 660.49 segundos. Como se observa en la Tabla 7, la optimización implementada ha resuelto eficazmente el problema en la etapa de fuzzificación, que deja de ser el componente crítico del sistema. No obstante, la reducción en el tiempo global de ejecución ha sido menor a la esperada. Pese a la gestión eficiente de memoria implementada, la lógica secuencial en Python sigue penalizando el rendimiento global.

Por tanto, se concluye que, para lograr una reducción drástica en los tiempos totales, la siguiente fase de optimización debe centrarse en vectorizar y compilar las funciones críticas del ciclo evolutivo: el cálculo del soporte (cobertura de reglas) y el Mecanismo de Razonamiento Difuso (FRM) dentro de la evaluación de la población.

A continuación, se presentan los resultados obtenidos para el resto de los conjuntos de datos en la tabla 8. Si bien el comportamiento observado en Wisconsin (donde la reducción del tiempo global es marginal) es extrapolable a estos casos, se considera relevante documentar las métricas de esta etapa intermedia para establecer una comparación al final de este proyecto. Dada la ineficiencia computacional en esta versión, la tabla se restringe exclusivamente a aquellos datasets que lograron completar su ejecución en la versión anterior.

dataset	TA	xTA	TG	xTG	TT	xTT
wine	00:00:15:15	131,76	00:01:44:69	206,91	00:01:59:84	191,75
balance	00:00:03:98	56,05	00:20:22:91	254,40	00:20:26:90	251,36
contraceptive	excesivo	----	Excesivo	----	excesivo	----
newthyroid	00:00:00:27	12,52	00:01:16:99	179,04	00:01:17:27	169,07

thyroid	00:02:08:77	186,35	01:05:41:63	194,21	01:07:50:41	193,75
post-operative	00:00:02:62	59,69	00:00:15:96	86,75	00:00:18:59	79,11
splice	excesivo	----	Excesivo	----	excesivo	----
iris	00:00:00:09	5,72	00:00:19:73	87,31	00:00:19:83	78,69
glass	00:00:09:83	118,47	00:07:11:32	242,04	00:07:21:16	235,66
tic-tac-toe	excesivo	----	Excesivo	----	excesivo	----
wisconsin	00:00:02:07	41,46	00:08:23:05	318,58	00:08:25:12	308,19
breast	00:00:13:14	126,37	00:01:23:71	144,09	00:01:36:86	138,18
shuttle	excesivo	----	excesivo	----	excesivo	----
saheart	00:00:22:75	127,80	00:21:53:29	239,82	00:22:16:04	235,97
magic	excesivo	----	excesivo	----	Excesivo	----
banana	00:00:01:27	22,05	00:45:31:64	251,76	00:45:32:92	249,72
phoneme	00:00:24:03	96,90	01:43:24:70	271,33	01:43:48:73	269,21
page-blocks	excesivo	----	excesivo	----	excesivo	----
texture	excesivo	----	excesivo	----	excesivo	----
mushroom	excesivo	----	excesivo	----	excesivo	----
satimage	excesivo	----	excesivo	----	excesivo	----

Tabla 8: datos de la versión Python con mascara de datos en formato horas:minutos:segundos:milisegundos para las mediciones temporales

Una vez más, y como se realizo en el experimento anterior, adjunto una tabla resumen (tabla 9) que complementa a la tabla general.

Datasets	Métrica	Tiempo medio (HH: MM: SS)	veces java	Datasets validos
DS con 3 clases	TA	00:00:25	75	6 de 8
	TG	00:14:56	168	6 de 8
	TT	00:15:22	160	6 de 8
DS con 9/10 atributos	TA	00:00:11	103	4 de 7
	TG	00:09:42	236	4 de 7
	TT	00:09:54	229	4 de 7
DS con 5300/6400 ejemplos	TA	00:00:12	59	2 de 6
	TG	01:14:28	261	2 de 6
	TT	01:14:40	259	2 de 6

Tabla 9: Resumen de los tiempos de ejecución en Python por grupos de datasets

5.3. Optimización mediante Vectorización: Eliminación de Bucles en Soporte y FRM

En esta tercera versión en Python, la primera de las optimizaciones se ha centrado en un rediseño completo del método evaluate de la base de reglas y del Mecanismo de Razonamiento Difuso (FRM), eliminando la lógica iterativa de la versión anterior.

Se ha sustituido el flujo de control explícito por operaciones matriciales masivas mediante NumPy, transformando el cálculo de grados de matching y la agregación de clases en operaciones de álgebra lineal sobre matrices tridimensionales (Ejemplos x Reglas x Etiquetas). Esto permite realizar la inferencia de todo el conjunto de datos de manera simultánea (batch processing). El procedimiento implementado proyecta los datos pre-fuzzificados y las máscaras de activación de las reglas en estructuras tridimensionales compatibles mediante técnicas de expansión de dimensiones. Esto permite calcular la compatibilidad de todas las reglas contra todos los ejemplos en una única operación de producto elemento a elemento. Finalmente, el mismo evaluate resuelve la inferencia de reglas.

La segunda de las mejoras se ha centrado en el algoritmo A priori, en concreto en el recálculo redundante de soportes. Se ha modificado la estructura de la clase Itemset para que actúe como un contenedor de vectores pre-calculados. La lógica se ha transformado de la siguiente manera:

- **Cálculo inicial:** En el primer nivel, se obtienen y almacenan en memoria los vectores de grados de pertenencia completos para cada ítem candidato.
- **Propagación:** Al generar un nuevo candidato (combinación de ítems), el algoritmo ya no recorre el dataset original. En su lugar, recupera los vectores almacenados en los itemsets padres y realiza una intersección vectorial directa (producto).

Este cambio reduce drásticamente la complejidad del algoritmo, eliminando la dependencia del número de variables en el bucle interno y limitando el coste computacional a operaciones algebraicas simples sobre vectores de longitud N.

Para cuantificar el impacto de estas optimizaciones, se ha repetido el análisis de perfilado sobre el dataset Wisconsin. Como se observa en la Tabla 10, las funciones que anteriormente saturaban el procesador han desaparecido de los primeros puestos o han reducido drásticamente su peso relativo, confirmando el éxito de la vectorización.

Función (archivo)	N.º de Llamadas	Tiempo	% del T. Total
evaluate (RuleBase.py)	10,382	55.30 s	40.00%
numpy.repeat (Método interno)	20,764	43.57 s	31.51%
StringRep (Individual.py)	7,400	8.09 s	5.85%
evaluate (Individual.py)	10,380	5.48 s	3.96%
FuzzificaEtiqueta (Fuzzy.py)	198,664	4.15 s	3.00%

Tabla 10: Funciones con mayor coste computacional (Dataset: Wisconsin) - V3

Al analizar estos datos, se observan cambios drásticos respecto a la versión anterior:

- **Desaparición de la lógica iterativa:** Las funciones degreeProduct y matching, que en la versión v2 consumían más del 70% del tiempo, han desaparecido de la tabla de tiempos significativos.
- **Nuevo cuello de botella:** El método evaluate de la base de reglas concentra ahora el 40% del tiempo. Sin embargo, no es algo que pueda optimizarse más (al menos empleando vectorización y máscaras de datos).
- **función repeat de numpy:** utilizada para expandir las matrices y hacerlas compatibles dimensionalmente, consume un 31.5% del tiempo total. Esto indica que la preparación y copia de datos en memoria para construir las matrices 3D se ha convertido en el nuevo factor limitante.

El tiempo total para Wisconsin se ha reducido considerablemente respecto a la versión anterior. A continuación, la Tabla 11 muestra los tiempos de ejecución globales para los datasets procesados. Es importante notar que, gracias a la eliminación de los bucles explícitos en Python, conjuntos de datos que anteriormente se clasificaban como "excesivos" o inviables ahora logran completar su ejecución dentro de márgenes razonables.

dataset	TA	xTA	TG	xTG	TT	xTT
wine	00:13:01	113,11	00:40:20	79,45	00:53:21	85,13
balance	00:03:39	47,81	03:44:74	46,75	03:48:13	46,73
contraceptive	01:17:36	154,42	33:18:95	68,63	34:36:31	70,07
newthyroid	00:00:21	9,68	00:24:91	57,93	00:25:12	54,98
thyroid	00:13:37	19,34	12:15:60	36,24	12:28:97	35,65
post-operative	00:02:43	55,42	00:02:87	15,64	00:05:31	22,62
splice	excesivo	----	excesivo	----	excesivo	----
iris	00:00:07	3,86	00:07:12	31,50	00:07:18	28,51
glass	00:00:89	58,89	02:12:87	74,56	02:17:76	73,59
tic-tac-toe	01:56:19	235,21	07:48:71	78,27	09:44:96	89,98
wisconsin	00:01:81	36,22	02:16:45	86,41	02:18:26	84,35
breast	00:12:91	124,13	00:33:43	57,54	00:46:34	66,11
shuttle	05:37:69	71,51	excesivo	----	excesivo	----
saheart	00:20:59	115,72	excesivo	----	excesivo	----
magic	19:35:27	202,00	excesivo	----	excesivo	----
banana	00:01:05	18,14	04:18:72	23,84	04:19:77	23,73
phoneme	00:20:52	82,77	25:48:31	67,70	26:08:84	67,80
page-blocks	00:44:09	113,06	50:07:95	114,59	50:52:04	114,47
texture	Excesivo	----	excesivo	----	Excesivo	----

mushroom	18:52:48	331,13	09:54:05	293,64	28:46:53	315,81
satimage	excesivo	----	Excesivo	----	Excesivo	----

Tabla 11: Resultados globales de la versión Python V3 (Vectorizada) en formato minutos:segundos:milisegundos para las mediciones temporales.

Como en los casos anteriores adjunto una tabla resumen (tabla 12) que complementa a la tabla general. Cabe destacar que algunos de los valores medios se han incrementado, pero esto es debido a que se tienen en cuenta nuevos datasets que tienen un peso muy elevado respecto al resto.

Datasets	Métrica	Tiempo medio	veces java	Datasets validos
DS con 3 clases	TA	00:00:15	57	7 de 8
	TG	00:07:13	48	7 de 8
	TT	00:07:29	49	7 de 8
DS con 9/10 atributos	TA	00:03:57	120	7 de 7
	TG	00:03:12	74	4 de 7
	TT	00:03:46	78	4 de 7
DS con 5300/6400 ejemplos	TA	00:04:59	136	4 de 6
	TG	00:22:32	124	4 de 6
	TT	00:27:31	130	4 de 6

Tabla 12: Resumen de los tiempos de ejecución en Python por grupos de datasets

Los resultados obtenidos demuestran que la vectorización es una estrategia altamente efectiva para reducir las limitaciones de rendimiento del intérprete de Python en tareas de minería de datos. Sin embargo, los datos obtenidos en el perfil del dataset wisconsin revelan una nueva barrera: al acelerar la lógica difusa, el peso de la ejecución ha recaído en la gestión de memoria (debido a la copia masiva de datos en operaciones como `numpy.repeat`) y funciones auxiliares con un gran número de operaciones matemáticas. Para superar estas nuevas limitaciones y acercarse al rendimiento de la implementación original en Java, la siguiente y última fase de optimización explorará el uso de Broadcasting implícito y la compilación JIT mediante la librería Numba.

5.4. Compilación JIT con Numba

Tras las mejoras de la versión anterior, se detectó que la gestión de memoria y el overhead del intérprete de Python (especialmente crítico en la ejecución iterativa de operaciones matemáticas simples) seguían limitando el rendimiento global. Para solucionar esto, se implementó la librería Numba, un compilador JIT que traduce funciones de Python a código máquina nativo optimizado en tiempo de ejecución, permitiendo alcanzar velocidades de procesamiento comparables a lenguajes de bajo nivel como C o Fortran. Para obtener métricas de rendimiento fieles a un entorno de producción, se ha implementado una fase de precalentamiento (warm-up), consistente en ejecutar las funciones críticas previamente. Esto asegura que los kernels estén ya compilados y alojados en la caché de instrucciones de la CPU, permitiendo medir

exclusivamente la velocidad de inferencia pura y facilitando una comparación equitativa con la implementación original en Java. En concreto, se han aplicado las siguientes mejoras:

- **Paralelización de bucles independientes:** Se identificaron las secciones críticas donde los cálculos no tienen dependencias entre iteraciones, como la evaluación de reglas (`compute_matchings`) o la fuzzificación de ejemplos (`fuzzificacion_total`). Se han sustituido los bucles estándar por `prange` (`Parallel Range`) bajo el decorador `@njit(parallel=True)`. Esto permite que el algoritmo distribuya la carga de trabajo entre todos los núcleos disponibles del procesador, logrando una ejecución simultánea real.
- **Compilación de funciones matemáticas con `@njit`:** Las funciones que realizan cálculos intensivos y repetitivos, como el cálculo del `WrAcc` en la fase A Priori, los operadores de cruce `xPC_BLX` o la distancia de Hamming, fueron decoradas con `@njit(fastmath=True)`. Esto compila el código Python a código máquina optimizado en tiempo de ejecución, permitiendo que operaciones matemáticas complejas se ejecuten a una velocidad comparable a C o Fortran.
- **Implementación de clases completas en bajo nivel:** se ha reescrito la clase `MTwister` (generador de números aleatorios) utilizando el decorador `@jitclass`. Esto convierte el objeto entero en una estructura compilada, permitiendo que las funciones de mutación y cruce generen millones de números aleatorios directamente dentro del entorno optimizado sin volver a Python.
- **Uso de Broadcasting Implícito:** Para solucionar el consumo excesivo de memoria RAM de la versión anterior (V3) debido a las matrices trisimensionales en la evaluación de conjuntos de reglas, se implementó el `broadcasting implícito` mediante la indexación con `np.newaxis`. Esta técnica permite operar matrices de distintas dimensiones (`Ejemplos vs Reglas`) alineándolas virtualmente, sin necesidad de duplicar físicamente los datos en memoria.
- **Optimizaciones puntuales de funciones auxiliares:** Finalmente, se reescribieron funciones auxiliares que, aunque simples, se ejecutaban muchas veces, como `obtenerLabelsUsadas` y `StringRep`. Estas funciones se adaptaron para ser compatibles con Numba, eliminando operaciones lentas de manejo de cadenas y listas de Python en favor de arrays de caracteres y operaciones numéricas directas, agilizando la gestión de individuos dentro del bucle evolutivo.

Para cuantificar el impacto de la compilación JIT y la paralelización, se ha sometido al sistema a un nuevo análisis de perfilado sobre el dataset `Wisconsin`, bajo las mismas condiciones que en las versiones anteriores. Los resultados, reflejados en la Tabla 13, evidencian una transformación radical en el comportamiento del algoritmo.

Función (archivo)	N.º de Llamadas	Tiempo	% del T. Total
evaluate (RuleBase.py)	10,382	1.98 s	83.8%
prefuzzyGA (RuleBase.py)	10,381	0.49 s	20.6%
decode (DataBase.py)	10,381	0.32 s	13.5%
numpy.unique (arraysetops.py)	10,382	0.18 s	7.5%
crossover (Population.py)	296	0.11 s	4.5%

Tabla 13: Funciones con mayor coste computacional (Dataset: Wisconsin) - V4.

Al observar los nuevos datos obtenidos por el perfil del dataset Wisconsin, podemos sacar las siguientes conclusiones:

- **Desaparición del cuello de botella de memoria:** La implementación con Numba, al utilizar bucles paralelos (prange) y acceso directo a memoria, ha eliminado completamente la necesidad de crear matrices intermedias masivas.
- **Aceleración de la evaluación de conjuntos de reglas:** Aunque la función evaluate sigue siendo la que ocupa el mayor porcentaje relativo (83.8%) en la V4, su tiempo absoluto se ha reducido considerablemente. Numba ha compilado las operaciones de producto-T y suma de votos a código máquina nativo, eliminando el overhead de la versión previa.
- **Funciones de gestión de datos:** Al reducirse el tiempo de cómputo matemático, funciones auxiliares de gestión de datos (como decode o prefuzzyGa) han ganado protagonismo en el perfil porcentual. No significa que sean lentas, sino que el grueso del algoritmo se volvió más rápido.
- **Aparición de numpy.unique:** puede deberse a que las operaciones de conjunto nativas de NumPy, son más lentas que código JIT paralelizado.

Como muchas de las funciones que han aparecido pueden continuarse mejorando con numba he realizado unos cambios extra:

- **Función de fuzzificación:** Se ha reescrito la función de fuzzificación para garantizar la independencia de datos en cada iteración, permitiendo así la paralelización efectiva de las operaciones. La clase Fuzzy ha pasado a actuar como un contenedor ligero, mientras que el proceso de cálculo se alimenta directamente de datos precalculados y optimizados en la Database.
- **Clase Rulebase:** Se ha mejorado la gestión interna de datos para reducir la sobrecarga de memoria. Esto incluye el almacenamiento en caché de las clases únicas y, crucialmente, la implementación de buffers temporales para evitar la asignación y liberación continua de memoria durante la ejecución. Asimismo, se

han reescrito las subrutinas críticas de la función evaluate (como el cálculo de compatibilidad y la predicción de clases) para explotar al máximo las capacidades de vectorización y compilación de Numba.

La implementación final empleando Numba ha logrado acelerar el código original en Python más de 250 veces en el dataset Wisconsin, alcanzando un rendimiento prácticamente idéntico al de la versión original en Java (1.99s vs 1.64s). Esto demuestra que es posible obtener un buen rendimiento en Python combinando estrategias de vectorización inteligente (broadcasting) y compilación JIT, manteniendo la flexibilidad del lenguaje para la experimentación científica. El rendimiento global de todos los datasets puede observarse en la tabla 14.

dataset	TA	xTA	TG	xTG	TT	xTT
wine	00:00:56	4,82	00:00:79	1,56	00:01:35	2,15
balance	00:00:03	0,35	00:01:81	0,38	00:01:84	0,38
contraceptive	00:00:35	0,69	00:07:83	0,27	00:08:18	0,28
newthyroid	00:00:01	0,59	00:01:58	3,66	00:01:59	3,47
thyroid	00:00:74	1,06	00:05:67	0,28	00:06:41	0,30
post-operative	00:00:06	1,37	00:00:93	5,05	00:00:99	4,21
splice	16:57:58	1,53	00:15:49	0,06	17:13:07	1,14
iris	00:00:01	0,30	00:00:63	2,79	00:00:64	2,52
glass	00:01:05	12,71	00:01:82	1,02	00:02:87	1,54
tic-tac-toe	00:00:39	0,78	00:01:34	0,22	00:01:73	0,26
wisconsin	00:00:01	0,28	00:01:10	0,69	00:01:11	0,68
breast	00:00:11	1,01	00:00:21	0,36	00:00:32	0,45
shuttle	00:02:31	0,45	02:45:07	1,16	02:47:38	1,14
saheart	00:00:14	0,81	00:02:16	0,39	00:02:30	0,41
magic	00:02:72	0,47	01:31:25	0,39	01:33:97	0,39
banana	00:00:01	0,09	00:04:67	0,43	00:04:68	0,43
phoneme	00:00:06	0,25	00:09:58	0,42	00:10:64	0,42
page-blocks	00:00:60	1,55	00:18:19	0,69	00:18:79	0,70
texture	03:01:00	2,07	01:04:26	0,54	04:05:26	1,19
mushroom	00:03:11	0,91	00:01:01	0,50	00:04:12	0,75
satimage	02:00:11	1,19	00:40:67	0,53	02:40:78	0,90

Tabla 14: Resultados globales de la versión Python V4 (Numba).

Como en los casos anteriores adjunto una tabla resumen (tabla 15) que complementa a la tabla general. En este caso, la tabla de medias cuenta con todos los datasets para extraer datos (como no pasaba en los casos anteriores), lo cual permite ver tiempos medios reales.

Datasets	Métrica	Tiempo medio (HH:MM:SS)	veces java	Datasets validos
DS con 3 clases	TA	00:02:07	1	8 de 8
	TG	00:00:04	1	8 de 8
	TT	00:02:11	1	8 de 8
DS con 9/10 atributos	TA	00:00:00	2	7 de 7
	TG	00:00:37	0	7 de 7
	TT	00:00:38	0	7 de 7
DS con 5300/6400 ejemplos	TA	00:00:50	1	6 de 6
	TG	00:00:23	0	6 de 6
	TT	00:01:14	0	6 de 6

Tabla 15: Resumen de los tiempos de ejecución en Python por grupos de datasets

En primer lugar, es relevante destacar que la mayor parte de los conjuntos de datos analizados presenta tiempos totales iguales o incluso inferiores a los de la versión original, lo que reafirma la efectividad de las técnicas de optimización empleadas y valida que Python puede alcanzar rendimientos competitivos cuando se combina con estrategias de optimización de bajo nivel.

En segundo lugar, los tiempos de ejecución se ven directamente influenciados por el uso de los 6 núcleos del procesador empleados en este experimento. En procesos paralelizados como la evaluación de reglas y la fuzzificación, el rendimiento mejora drásticamente conforme aumenta la carga de trabajo.

El número de ejemplos es uno de los factores más determinantes en el rendimiento relativo del algoritmo. En datasets con un volumen medio o alto de instancias (thyroid o banana), esta versión muestra un comportamiento increíblemente favorable. Por el contrario, en datasets muy pequeños (wine, iris o post-operative), el sistema tiende a ser más lento que el original. Este comportamiento se debe a que el coste asociado a la inicialización de estructuras y a la ejecución del proceso evolutivo no se ve compensado cuando el número de patrones es reducido.

El número de clases también influye de forma significativa en el rendimiento global. En problemas binarios (como tic-tac-toe o breast), el algoritmo presenta, en general, sus mejores resultados. En problemas con tres clases (resaltados en rojo), el comportamiento es más dependiente del tamaño del dataset. En datasets con un número elevado de clases (como glass, satimage o texture), el tiempo de ejecución es superior pero no en exceso.

El número de atributos es el último factor para tener en cuenta. Incluso en problemas de alta dimensionalidad (como splice o texture), se logran obtener resultados en tiempos razonables, evitando la degradación observada en versiones previas. La

vectorización y la compilación JIT permiten mantener el crecimiento del tiempo bajo control, incluso en espacios de búsqueda amplios.

5.5 Evaluación global del rendimiento

En la Tabla 11 se presenta el resumen definitivo de los datos de rendimiento obtenidos a lo largo del ciclo de vida del proyecto. Es importante destacar que, para esta evaluación global, se ha prescindido de las unidades de tiempo absoluto en favor de mostrar cuantas veces dura más el algoritmo en Python respecto a Java. Esta decisión metodológica se ha tomado para garantizar la independencia del hardware porque en ordenadores con características diferentes, los tiempos de ejecución serán diferentes, pero no debería serlo este valor.

El análisis de la tabla muestra una progresión clara en la eficiencia del código:

1. **Versiones iniciales (python₁ y python₂):** Los factores de coste son extremadamente altos (entre 81 a 342 veces más lento que Java), llegando incluso a la inviabilidad técnica en datasets complejos (marcados con "----"), donde el algoritmo no lograba finalizar en un tiempo razonable.
2. **Version intermedia (python₃):** Esta versión representa el primer cambio significativo en la tendencia. Si bien el rendimiento sigue lejos del original en Java (manteniéndose factores de ralentización entre 30 y 80 veces), se logra reducir en mayor medida el coste temporal respecto a las versiones iniciales.
3. **Versión final (python₄):** La versión final no solo consigue ejecutar todos los datasets sin excepciones, sino que en la mayoría de los datasets logra romper la barrera de la unidad, ejecutándose más rápido que el algoritmo original compilado en Java.

Finalmente, la última columna de la tabla 16 (v1_vs_v4) ofrece una perspectiva diferente: no se compara contra Java, sino que mide la evolución interna del desarrollo en Python. Este valor representa el factor de aceleración logrado entre la primera implementación ingenua y la versión final optimizada, con mejoras registradas de hasta 456 veces. Esta métrica es fundamental para validar el esfuerzo de ingeniería realizado, demostrando que, mediante el uso correcto de diferentes técnicas de optimización, es posible acelerar el código interpretado en varios órdenes de magnitud, haciendo viable el uso de Python en entornos de alto rendimiento.

dataset	python ₁	python ₂	python ₃	python ₄	v1_vs_v4
wine	195	191,75	85,13	2,15	90,70
balance	282	251,36	46,73	0,38	742,11
contraceptive	----	----	70,07	0,28	----
newthyroid	184	169,07	54,98	3,47	53,03

thyroid	228	193,75	35,65	0,30	760,00
post-operative	152	79,11	22,62	4,21	36,10
splice	----	----	----	1,14	----
iris	81	78,69	28,51	2,52	32,14
glass	257	235,66	73,59	1,54	166,88
tic-tac-toe	----	----	89,98	0,26	----
wisconsin	329	308,19	84,35	0,68	483,82
breast	273	138,18	66,11	0,45	606,67
shuttle	----	----	----	1,14	----
saheart	285	235,97	----	0,41	695,12
magic	----	----	----	0,39	----
banana	323	249,72	23,73	0,43	751,16
phoneme	342	269,21	67,8	0,42	814,29
page-blocks	----	----	114,47	0,70	----
texture	----	----	----	1,19	----
mushroom	----	----	315,81	1,2	----
satimage	----	----	----	1,54	----

Tabla 16: recopilación de datos de todas las versiones

Para visualizar de manera más clara la evolución del rendimiento, se han incluido las figuras 12 y 13. Ambas muestran cinco datasets que contienen datos de todos los experimentos realizados. La selección de estos datasets permite observar casos en los que el algoritmo final presenta un rendimiento superior, similar o inferior al de su versión en Java, con el objetivo de mantener transparencia en los resultados. He decidido incluir ambas figuras: la primera utiliza escala lineal para apreciar la magnitud de la mejora en las diferentes versiones de python, mientras que la segunda emplea escala logarítmica para resaltar las diferencias entre la ejecución en Java y la versión final en Python.

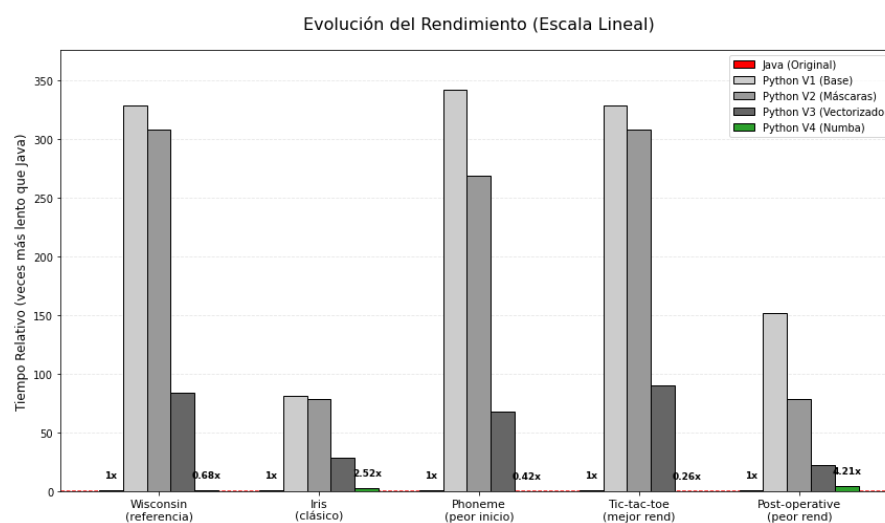


Figura 12: representación gráfica de la evolución del rendimiento

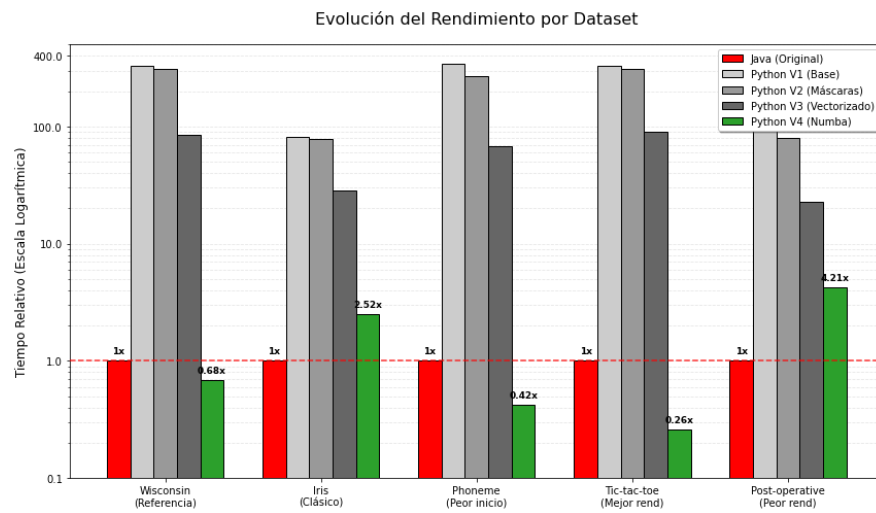


Figura 13: representación gráfica con escala logarítmica

6. Conclusiones y trabajo futuro

6.1 Conclusiones Generales

El desarrollo de este proyecto ha permitido validar la viabilidad de trasladar algoritmos complejos de minería de datos desde un entorno compilado (Java) a un entorno interpretado (Python) sin sacrificar el rendimiento, siempre que se apliquen las técnicas de optimización adecuadas. Los resultados experimentales demuestran que, si bien una traducción directa conlleva una penalización temporal inasumible, una reconstrucción cuidadosa del código permite alcanzar niveles de eficiencia competitivos.

En concreto, la implementación final basada en la compilación JIT (Just-In-Time) con Numba, combinada con estrategias de vectorización, broadcasting implícito y paralelización de bucles, transformó radicalmente el comportamiento del algoritmo. Estas mejoras lograron acelerar el código original en Python más de 250 veces en casos representativos como el dataset Wisconsin, alcanzando un rendimiento prácticamente idéntico (y en ocasiones superior) a la implementación original en Java. Esto confirma que Python, apoyado en herramientas de bajo nivel, es capaz de superar sus limitaciones de gestión de memoria y ejecución iterativa.

No obstante, el análisis detallado del rendimiento revela que la eficiencia de la solución no es lineal, sino que depende críticamente de las características del problema y del hardware:

- **Volumen de datos:** La solución optimizada muestra su mayor potencial en datasets de tamaño medio y grande, donde la paralelización en múltiples núcleos compensa el coste computacional. Por el contrario, en datasets pequeños (como Wine o Iris), el overhead asociado a la inicialización de estructuras y a la compilación JIT penaliza el tiempo total, haciendo que el sistema resulte más lento que la versión original. En estos casos, el tiempo de ejecución del algoritmo es parecido al requerido para la fase de compilación y preparación del código optimizado, lo que impide amortizar el coste de dichas optimizaciones.
- **Complejidad del problema:** El sistema se comporta de manera óptima en problemas binarios y pocos atributos. Aunque el aumento en el número de clases o atributos incrementa el tiempo de ejecución, las técnicas de vectorización aplicadas han logrado mantener este crecimiento bajo control, evitando la explosión combinatoria observada en las primeras versiones.

En conclusión, este trabajo demuestra que es posible unificar la flexibilidad de Python para la experimentación científica con la eficiencia necesaria para procesar grandes volúmenes de datos, siempre que se comprendan y gestionen los compromisos entre el tiempo de compilación y la velocidad de ejecución pura.

6.2 Líneas de Trabajo Futuro

A pesar de los resultados alcanzados, el proyecto abre nuevas vías de investigación y mejora. A continuación, se detallan las principales líneas de actuación propuestas para la evolución de la herramienta:

- **Refactorización hacia un Diseño Orientado a Datos (Data-Oriented Design) [57]:** Aunque la versión final (Python_4) ha optimizado gran parte del flujo, aún persisten estructuras heredadas del paradigma de Orientación a Objetos (POO) original de Java. Por ello, se propone eliminar la dependencia de objetos complejos para representar individuos o reglas y sustituirlos por estructuras de datos contiguas, como arrays de NumPy o estructuras de bajo nivel, con el objetivo de reducir el coste asociado a la gestión de punteros y metadatos en Python.
- **Mejora de las métricas de rendimiento mediante modificaciones en el algoritmo:** Se podría intervenir en la lógica interna del algoritmo para elevar la calidad de las predicciones sin comprometer el rendimiento obtenido. Las posibles partes del algoritmo donde se podría intervenir son:
 - **Optimización de la Poda en Apriori:** Refinar los criterios estadísticos para descartar itemsets irrelevantes de forma más temprana, conservando solo aquellos con mayor poder discriminante.
 - **Nuevos Operadores Genéticos:** Diseñar estrategias genéticas (fitness, cruce, selección...) más agresivas o adaptativas que favorezcan una exploración más eficiente del espacio de búsqueda, evitando óptimos locales.
 - **Inducción y Reducción de Reglas:** Revisar los mecanismos de creación de reglas para generar conjuntos de reglas más compactos y generalizables, reduciendo el riesgo de sobreajuste.
- **Búsqueda de Hiperparámetros:** Actualmente, la configuración de parámetros críticos como el soporte mínimo, la confianza, la profundidad de los árboles o el tamaño de la población se realiza de forma estática o manual, lo que limita la adaptabilidad del algoritmo a distintos datasets. La propuesta consiste en desarrollar un mecanismo capaz de explorar automáticamente el espacio de hiperparámetros para identificar la configuración óptima en cada caso, mejorando así la eficiencia del algoritmo y la calidad de las soluciones generadas.
- **Integración en Sistemas de Aprendizaje por Conjuntos (Ensemble Learning):** Dada la naturaleza del algoritmo, este puede beneficiarse de la cooperación con otros modelos predictivos. Se podría estandarizar la interfaz de la herramienta

para permitir su inclusión en pipelines de votación o stacking junto a otros clasificadores (como Random Forest o SVM).

- **Preprocesamiento Avanzado de Datos:** La calidad de la entrada determina el límite superior del rendimiento del algoritmo. Por ello, incorporar una etapa previa de limpieza y transformación automática permitiría un algoritmo más robusto frente a datasets con ruido, datos faltantes, outliers...
- **Optimización de módulos críticos con Cython:** Convertir las secciones más intensivas en cálculo del código Python a Cython permitiría mejorar significativamente el rendimiento (similar a Numba, ya que ambos generan código en C) y crear librerías reutilizables, manteniendo la compatibilidad con Python y con librerías como NumPy.

8. Bibliografía y referencias

- [1] M. Jordan y T. Mitchell, "Machine learning: Trends, perspectives, and prospects," *Science*, vol. 349, no. 6245, pp. 255–260, 2015. Disponible: <https://www.science.org/doi/10.1126/science.aaa8415>
- [2] F. Herrera, "Genetic fuzzy systems: Taxonomy, current research trends and prospects," *Evolutionary Computation*, vol. 12, no. 2, pp. 123–138, 2008. Disponible: <https://link.springer.com/article/10.1007/s12065-007-0001-5>
- [3] A. L. Fernández Hilario, "*Sistemas de clasificación basados en reglas difusas lingüísticas aplicadas a problemas con clases no balanceadas*", Tesis doctoral, Dept. de Ciencias de la Computación e Inteligencia Artificial, Universidad de Granada, Granada, España, Mar. 2010. Disponible en: <http://hdl.handle.net/10481/4925>
- [4] J. Alcalá-Fdez, R. Alcalá, y F. Herrera, "A fuzzy association rule-based classification model for high-dimensional problems with genetic rule selection and lateral tuning," *IEEE Transactions on Fuzzy Systems*, vol. 19, no. 5, pp. 857–872, 2011. Disponible: <https://ieeexplore.ieee.org/document/5756477>
- [5] IBM, "Explainable AI (XAI)," IBM THINK, 2025. Disponible: <https://www.ibm.com/es-es/think/topics/explainable-ai>
- [6] "Introduction to Classification," Scikit-learn Documentation. Disponible: https://scikit-learn.org/stable/supervised_learning.html
- [7] "Machine Learning: los orígenes y la evolución," FutureSpace, 2023. [En línea]. Disponible en: <https://www.futurespace.es/machine-learning-los-origenes-y-la-evolucion>
- [8] IBM. (s. f.). ¿Qué es la Inteligencia Artificial? Disponible: <https://www.ibm.com/es-es/think/topics/artificial-intelligence>
- [9] M. M. Rashid, A. Shafie, y M. Imran, "A review of machine learning algorithms: Concepts, applications, and trends," in **Proc. 4th Int. Conf. Control, Commun. Comput. Inf. Technol. (ICCUBE)**, Pune, India, 2018, pp. 1–6, doi: 10.1109/ICCUBE.2018.8697857. Disponible en: <https://ieeexplore.ieee.org/abstract/document/8697857>
- [10] Universidad Nacional Autónoma de México, "Lógica Difusa" *INTAR*. Disponible: https://virtual.cuautitlan.unam.mx/intar/?page_id=997
- [11] "Aprendizaje supervisado y no supervisado," Universidad Europea. Disponible: <https://universidadeuropea.com/blog/aprendizaje-supervisado-no-supervisado/>

- [12] "Aprendizaje supervisado y no supervisado," Health Data Miner. Disponible: <https://healthdataminer.com/data-mining/aprendizaje-supervisado-y-no-supervisado/>
- [13] "Supervised vs. Unsupervised Learning," Google Cloud. Disponible: <https://cloud.google.com/discover/supervised-vs-unsupervised-learning?hl=es>
- [14] "Modelos de clasificación", IBM Think, 2024. Disponible en: <https://www.ibm.com/es-es/think/topics/classification-models>
- [15] "Overfitting vs. Underfitting", IBM. Disponible en: <https://www.ibm.com/think/topics/overfitting-vs-underfitting>
- [16] C. M. Pineda Pertuz, Aprendizaje automático y profundo en Python: Una mirada hacia la inteligencia artificial. Madrid, España: Ra-Ma Editorial, 2021.
- [17] "A 5 minute guide to Numba". Numba User Manual. Disponible en: <https://numba.readthedocs.io/en/stable/user/5minguide.html>
- [18] W. Ertel, Introduction to Artificial Intelligence, 2nd ed. Cham, Switzerland: Springer International Publishing AG, 2018.
- [19] IBM, "¿Qué es el aprendizaje por refuerzo?", *IBM Think Blog*, 2024. Disponible en: <https://www.ibm.com/es-es/think/topics/reinforcement-learning>
- [20] R. O. Duda, P. E. Hart y D. G. Stork, *Pattern Classification*, 2.^a ed., Wiley-Interscience, 2000.
- [21] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002. Disponible en: <https://www.jair.org/index.php/jair/article/view/10302>
- [22] P. E. Hart, "The condensed nearest neighbor rule," *IEEE Trans. Inf. Theory*, vol. 14, no. 3, pp. 515–516, May 1968. Disponible en: <https://ieeexplore.ieee.org/document/1054155>
- [23] I. Tomek, "Two modifications of CNN," *IEEE Trans. Syst., Man, Cybern.*, vol. 6, no. 11, pp. 769–772, Nov. 1976. Disponible en: <https://ieeexplore.ieee.org/document/4309452>
- [24] M. Kubat and S. Matwin, "Addressing the curse of imbalanced training sets: a case study in fraud detection," in *Proc. 14th Nat. Conf. Artif. Intell. (AAAI-97)*, 1997, pp. 169–174. Disponible en: <https://sci2s.ugr.es/keel/pdf/specific/congreso/kubat97addressing.pdf>

- [25] V. López, A. Fernández, S. García, and F. Herrera, "Learning from imbalanced data: open challenges and future directions," *Progress in Artificial Intelligence*, vol. 2, pp. 1–13, 2013. Disponible en: <https://link.springer.com/article/10.1007/s13748-016-0094-0>
- [26] B. Ozenne, F. Subtil, and D. Maucourt-Boulch, "The precision–recall curve overcame the optimism of the receiver operating characteristic curve in rare diseases," *Journal of Clinical Epidemiology*, vol. 68, no. 8, pp. 855–859, 2015, Elsevier. Disponible en: <https://www.sciencedirect.com/science/article/pii/S0895435615001067>
- [27] H. Bustince, E. Barrenechea, M. Pagola, J. Fernández, Z. Xu, B. Bedregal, J. Montero, H. Hagrass, F. Herrera and B. De Baets, "A historical account of types of fuzzy sets and their relationships," *IEEE Transactions on Fuzzy Systems*, vol. 24, no. 1, pp. 179–194, 2016, doi: 10.1109/TFUZZ.2015.2451692. Disponible en: <https://digibug.ugr.es/bitstream/handle/10481/62847/A%20historical%20account%20of%20types%20of%20fuzzy%20sets%20and%20their.pdf;jsessionid=1B4A8D29795DE80607FC418EBA60AA83?sequence=1>
- [28] F. Sancho, "Introducción a la Lógica Difusa," Departamento de Ciencias de la Computación e Inteligencia Artificial, Universidad de Sevilla. Disponible en: https://www.cs.us.es/~fsancho/Blog/posts/Introduccion_logica_Difusa.md.html.
- [29] J. Ferrer, "Lógica difusa en IA: Principios, Aplicaciones y Guía de Implementación en Python," DataCamp. Disponible en: <https://www.datacamp.com/es/tutorial/fuzzy-logic-in-ai>
- [30] M. M. Gupta, "Theory of T-norms and fuzzy inference methods," *Fuzzy Sets and Systems*, vol. 40, no. 3, pp. 431–450, 1991, doi:10.1016/0165-0114(91)90171-L. Disponible en: <https://www.sciencedirect.com/science/article/pii/016501149190171L>
- [31] H. Ishibuchi y T. Nakashima, "Effect of rule weights in fuzzy rule-based classification systems," *IEEE Transactions on Fuzzy Systems*, vol. 9, no. 4, pp. 506–515, Aug. 2001. Disponible en: https://ci-lab-omu.github.io/assets/paper/pdf_file/fuzzy/Effect_of_Rule.pdf
- [32] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor, MI, USA: Univ. Michigan Press, 1975. Disponible en: http://repo.darmajaya.ac.id/3794/1/Adaptation%20in%20Natural%20and%20Artificial%20Systems_%20An%20Introductory%20Analysis%20with%20Applications%20to%20Biology%2C%20Control%2C%20and%20Artificial%20Intelligence%20%28A%20Bradford%20Book%29%20%28%20PDFDrive%20%29.pdf
- [33] B. B. Benuwa, B. Ghansah, D. K. Wornyo, and S. A. Adabunu, "A comprehensive review of particle swarm optimization," *International Journal of Engineering Research in Africa*, vol. 23, pp. 141–161, 2016. Disponible en:

https://www.researchgate.net/publication/301272239_A_Comprehensive_Review_of_Particle_Swarm_Optimization#:~:text=Abstract,in%20the%20future%20are%20proposed.

[34] E. Rashedi, H. Nezamabadi-pour y S. Saryzdi, «GSA: A Gravitational Search Algorithm.,» *Information Sciences*, vol. 179, pp. 2232-2248, 2009. Disponible en: <https://www.sciencedirect.com/science/article/pii/S0020025509001200>

[35] J. I. Hidalgo Pérez y C. Cervigón Rückauer, “Una revisión de los algoritmos evolutivos y sus aplicaciones,” *Enlaces: revista del CES Felipe II*, no. 2, 2004, ISSN: 1695-8543. Disponible en: https://www.researchgate.net/profile/Ignacio-Hidalgo/publication/277274186_Una_revision_de_los_algoritmos_evolutivos_y_sus_aplicaciones/links/55edb76b08aedecb68fb4b79/Una-revision-de-los-algoritmos-evolutivos-y-sus-aplicaciones.pdf

[36] M. Gestal, D. Rivero, J. R. Rabuñal, J. Dorado, y A. Pazos, “Introducción a los Algoritmos Genéticos y la Programación Genética,” [En línea]. Disponible en: https://www.academia.edu/35232151/Introducci%C3%B3n_a_los_Algoritmos_Gen%C3%A9ticos_y_la_Programaci%C3%B3n_Gen%C3%A9tica

[37] J. R. Koza, *Genetic Programming: On the Programming of Computers by Means of Natural Selection*, Cambridge, MA, USA: MIT Press, 1992. Disponible en: <https://www.genetic-programming.com/jkpdf/sciournallong.pdf>

[38] R. Storn and K. Price, “Differential Evolution – A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces,” *Journal of Global Optimization*, vol. 11, no. 4, pp. 341–359, 1997. https://www.metabolic-economics.de/pages/seminar_theoretische_biologie_2007/literatur/schaber/Storn1997JGlobOpt11.pdf

[39] D. M. Ortiz, J. D. Velásquez H. y P. Jaramillo, “Evolution strategies as an option for optimizing non-linear functions with restrictions,” *Rev. ing. univ. Medellín*, vol. 10, no. 18, Jan.–June 2011. Disponible en: http://www.scielo.org.co/scielo.php?script=sci_arttext&pid=S1692-33242011000100013#:~:text=Estrategias%20de%20evoluci%C3%B3n%20es%20una,%20soluciones%20es%20no%20restringido.

[40] A. Shukla, H. M. Pandey, y D. Mehrotra, “Comparative review of selection techniques in genetic algorithm,” in *Proc. 2015 Int. Conf. Futuristic Trends on Computational Analysis and Knowledge Management (ABLAZE)*, Greater Noida, India, 25–27 Feb. 2015. Disponible en: <https://ieeexplore.ieee.org/document/7154916>

[41] D. E. Goldberg, *Genetic Algorithms in Search, Optimization and Machine Learning*, Addison-Wesley, 1989. Disponible en:

https://www2.fiiit.stuba.sk/~kvasnicka/Free%20books/Goldberg_Genetic_Algorithms_in_Search.pdf

[42] D. H. Wolpert y W. G. Macready, "No free lunch theorems for optimization," *IEEE Transactions on Evolutionary Computation*, vol. 1, no. 1, pp. 67–82, Apr. 1997, doi: 10.1109/4235.585893. Disponible en: <https://ieeexplore.ieee.org/document/585893>

[43] R. Agrawal and R. Srikant, "Fast algorithms for mining association rules," in *Proc. 20th Int. Conf. Very Large Data Bases (VLDB)*, Santiago de Chile, 1994, pp. 487–499. Disponible en: <https://www.vldb.org/conf/1994/P487.PDF>

[44] B. Kavšek and N. Lavrač, "Apriori-SD: Adapting association rule learning to subgroup discovery," *Applied Artificial Intelligence*, vol. 20, no. 7, pp. 543–583, 2006. Disponible en: https://link.springer.com/chapter/10.1007/978-3-642-31951-8_11

[45] F. Herrera y L. Martínez, "A 2-tuple fuzzy linguistic representation model for computing with words," *IEEE Transactions on Fuzzy Systems*, vol. 8, no. 6, pp. 746–752, 2000. Disponible en: <https://ieeexplore.ieee.org/document/890332>

[46] DataScientest, "Cross-Validation: definición e importancia en Machine Learning", DataScientest. Disponible en: <https://datascientest.com/es/cross-validation-definicion-e-importancia>

[47] R. Fabian, "Data-Oriented Design", 8th October 2018. Disponible en: <https://www.dataorienteddesign.com/dodbook.pdf>

[48] "KEEL: Dataset Repository," Sci2S, Universidad de Granada. Disponible en: <https://sci2s.ugr.es/keel/datasets.php>